



คู่มือและเอกสารทางเทคนิคฉบับสมบูรณ์ (Comprehensive Technical & Operational Manual)

ระบบควบคุมการผลิต xMixing Control Application

รหัสเอกสาร (Document No.)	MAN-SYS-MIX-001
แก้ไขครั้งที่ (Revision)	01 (Master Compilation)
วันที่บังคับใช้ (Effective Date)	25 มิถุนายน 2026
หน่วยงาน (Department)	ฝ่ายวิศวกรรม, ไอที และฝ่ายผลิต

test

Hybrid Architecture & Full Recipe Workflow

This document outlines the **Hybrid Communication Model** (CQRS) combined with the **Full Recipe Array (DB1511)** concept for the Mixing Control application.

1. System Architecture Diagram

```
```text (Mermaid Diagram) graph TD subgraph Frontend [Vue.js Tablet UI] UI[Vue Components] MQTT_Sub[useMQTT.ts] API_Call[fetch / axios] end
```

```
subgraph Backend [Python FastAPI] DB[(MySQL Database)] API[FastAPI Routes] Snap7[python-snap7] end
```

```
subgraph Middleware [Telemetry Bridge] NR[Node-RED] MQ[RabbitMQ / MQTT] end
```

```
subgraph PLC [Siemens S7-1200] DB1511[DB1511: Full Recipe Array] DB1512[DB1512: Live Telemetry] DB1513[DB1513: Handshake] end
```

```
%% Telemetry Flow (Fast & Visual) DB1512 -->|Read| NR NR -->|Publish| MQ MQ -->|Subscribe| MQTT_Sub MQTT_Sub --> UI
```

```
%% Command Flow (Strict & Transactional) UI ==>|1. HTTP POST /start-batch| API_Call API_Call ==> API API ==>|2. Validate & Read| DB API ==>|3. Write Full Array| Snap7 Snap7 ==>|Direct S7 Write| DB1511
```

```
%% Handshake Feedback (Database Sync) DB1513 -->|Read End-of-Step| Snap7 Snap7 ==>|4. Update Logs| DB
```

---

### ## 2. Step-by-Step App Workflow

- ### Phase A: Starting the Batch (The Command)
- Operator Action:** The operator scans a barcode and clicks "Start Production" on the Vue.js tablet.
  - API Request:** The frontend sends an HTTP `POST /api/production/start` to FastAPI.

3. **Database Assembly:** FastAPI queries `v\_sku\_complete` to get all 20 steps for that specific SKU.
4. **Data Packing:** FastAPI uses `plc\_interface.py` to pack all 20 steps into the binary array structure required by `type\_FullRecipe`.
5. **Direct PLC Write:** FastAPI uses `snap7` to write the entire binary array directly to `DB1511` and sets `Cmd\_LoadRecipe = True`.
6. **Confirmation:** FastAPI responds to the frontend with `200 OK`. (If the PLC is offline, FastAPI blocks the start and returns a 500 error).

### ### Phase B: Mixing Execution (The Telemetry)

1. **PLC Takes Over:** The PLC sees `Cmd\_LoadRecipe = True`, loads the array, and begins executing `Active\_Step = 1`.
2. **Telemetry Streaming:** Node-RED continuously reads `DB1512` every 500ms and publishes to MQTT.
3. **UI Updates:** The Vue.js frontend receives the MQTT payload and updates the temperature gauges, active row highlighting, and timer *instantly*.

### ### Phase C: Step Completion (The Handshake)

1. **PLC Finishes Step:** The PLC finishes Step 1, pulses `DB1513.Step\_Complete`, and instantly moves to Step 2.
2. **Python Background Worker:** A background task in FastAPI (polling `DB1513` via snap7) detects the pulse.
3. **Database Logging:** FastAPI logs the final weight, temperature, and end-time of Step 1 into the SQL database.

---

## ## 📦 3. Required Application Modules

To build this, you need to structure your app modules like this:

### ### Python Backend (FastAPI)

```
* **`plc_interface.py` (Update Needed):**
 * Add a new Pydantic model for `DB1511_FullRecipe` and `DB1511_RecipeStep`.
 * Write the `.serialize()` method to convert the array of steps into S7 byte format.
* **`plc_service.py` (New):**
 * Use `python-snap7` to manage the TCP connection to the PLC.
 * Function: `write_recipe_to_plc(batch_id, steps_array)`
* **`worker_handshake.py` (New):**
 * An `asyncio` background loop that reads `DB1513` (Handshake) every 1 second and writes step completion data to the SQL database.
* **`routes/production.py`:**
 * Endpoint for `POST /start-batch` that triggers the PLC write.
```

### ### Node-RED (Middleware)

```
* **Remove DB1510 Logic:** Delete all nodes that currently write to DB1510. Node-RED is no longer responsible for sending recipes.
* **Keep DB1512 Logic:** Keep the S7-Read nodes for DB1512 and publish them to MQTT exactly as they are.
```

### ### ● Vue.js Frontend (x3101-0110-frontEnd)

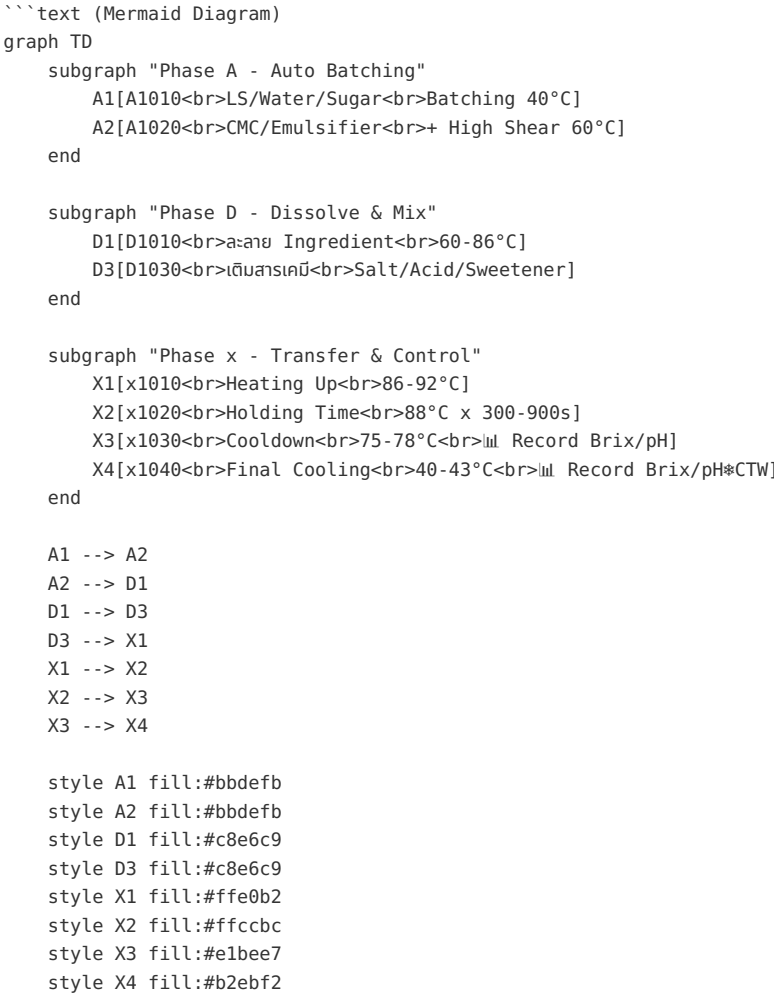
```
* **`x61-MixingControl.vue`:**
 * When pressing start/next, instead of publishing an MQTT message, call an Axios HTTP POST request to your FastAPI backend.
* **`useMQTT.ts`:**
 * Keep this exactly as it is! It will continue to listen to the telemetry stream to update the UI perfectly.
```

---

# 🏭 Process Control Concept – xMixing Production

## 1. Process Flow Overview

จากการวิเคราะห์ SKU ทั้ง 16 ตัวจริงจากฐานข้อมูล พบรูปแบบการผลิตดังนี้:



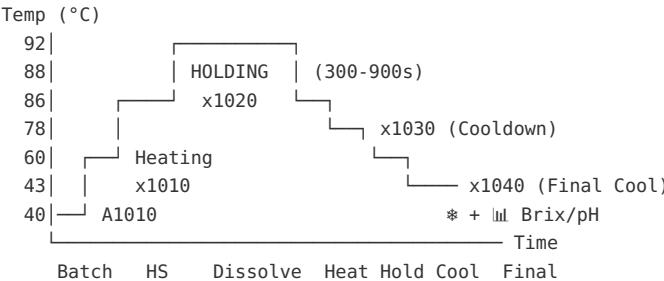
2. Temperature Zones (5 โซน)

Zone	Phase ID	อุณหภูมิ	Range	ระยะเวลา	วัตถุประสงค์
Mixing	A1010	40°C	-	ไม่กำหนด	ชั่งตวงวัตถุดิบหลัก (LS, Water, Sugar)
✂ High Shear	A1020	60°C	-	120-180s	ผสม CMC, Emulsifier + High Shear 800-1700 RPM
🔴 Heating	x1010	86-92°C	85-95°C	ไม่กำหนด	ให้ความร้อนถึงจุดพาสเจอร์ไรส์
Holding	x1020	88°C	86-90°C	300-900s	คงอุณหภูมิฆ่าเชื้อ (Pasteurization)

\* x1030→x1040 78→43°C 40-80°C ไม่กำหนด ลดอุณหภูมิ + เติม Flavour + QC Check

Cooldown

Temperature Profile Chart:



3. Motor Control (2 ระบบ)

3.1 Agitator (ใบกวนหลัก)

สถานะ	RPM	ใช้ในขั้นตอน
Low Speed	30 RPM	D1010, D1030 — ช่วงเติมวัตถุดิบละลาย
Normal Speed	60 RPM	A1020 — ช่วง High Shear ทำงาน
High Speed	80 RPM	A1010, x1010-x1040 — ช่วง Batch/Heat/Hold/Cool

3.2 High Shear (เครื่องบดละเอียด)

สถานะ	RPM	ใช้เมื่อ
OFF	0	ส่วนใหญ่ของกระบวนการ
Medium	800	เติมของเหนียว (Emulsifier, Textaid)
High	1700	เติม CMC, สารทำให้ข้น

[!!IMPORTANT] High Shear ทำงานเฉพาะ Phase **A1020** เท่านั้น! และมี **timer control** (120-180 วินาที)

4. Recording & QC Checkpoints

4.1 ค่าที่ต้อง Record ตลอด (ทุก Step)

ค่า	Field	บันทึก	ประเภท
อุณหภูมิ	qc_temp	ทุก Step	Continuous
Steam Pressure	record_steam_pressure	ทุก Step	Continuous

4.2 ค่าที่ Record เฉพาะบาง Step (QC Checkpoint)

คำ	Field	เมื่อไหร่	Phase ที่พบ
* CTW			
(Cooling Tower Water)	record_ctw	ช่วง Final Cooling เท่านั้น	x1040
๗ Brix	operation_brix_record	ช่วง Cooldown + Final	x1030, x1040
pH	operation_ph_record	ช่วง Cooldown + Final	x1030, x1040

### 4.3 QC Checkpoint Summary (จาก Blue Lemon Freshy 32 steps)

Step 1-23: ไม่มี Brix/pH record → เป็นขั้นตอนผลิตปกติ  
Step 24 (x1030 Cooldown 78°C): ๗ Brix    pH    ← QC Check #1  
Step 25-31: ไม่มี  
Step 32 (x1040 Final 43°C): ๗ Brix    pH    \* CTW    ← QC Check #2 (Final)

## 5. Data Flow: Recipe → PLC → Record

sequenceDiagram participant DB as Central DB participant HMI as x31 Dashboard participant PLC as PLC Controller participant REC as Production Record DB

Note over HMI: Operator clicks START  
HMI->>PLC: 📄 Download Recipe (ALL steps)

loop Each Step

PLC->>PLC: Execute step (dosing/heat/mix)  
PLC->>HMI: Telemetry (every 1s)<br>{temp, rpm, weight, timer}

alt Step has record flag

PLC->>HMI: Record Request<br>{actual\_temp, actual\_weight, actual\_brix, actual\_ph}  
HMI->>REC: Save Production Record

end

alt QC Checkpoint (x1030/x1040)

PLC->>HMI: QC Data<br>{brix\_actual, ph\_actual, ctw}  
HMI->>HMI: Compare vs SP (Setpoint)  
HMI->>REC: Save QC Record  
Note over HMI: PASS or x FAIL alert

end

end

PLC->>HMI: Batch Complete  
HMI->>REC: Save Batch Summary

---

## 6. Production Record Schema

### 6.1 Per-Step Record (บันทึกทุก Step)

```json

```
{
  "batch_id": "P260411-02-01-001",
  "step_no": 10,
  "phase": "p0010",
  "phase_id": "A1010",
  "re_code": "LS in Line",
```

```
"setpoint": {
  "require": 371.71,
  "temperature": 40.0,
  "agitator_rpm": 80.0,
  "high_shear_rpm": 0
},

"actual": {
  "weight": 371.85,
  "temperature": 40.2,
  "agitator_rpm": 79.5,
  "high_shear_rpm": 0,
  "steam_pressure": 2.5,
  "elapsed_time": 45
},

"tolerance": {
  "weight_ok": true,
  "temp_ok": true
},

"timestamp_start": "2026-04-15T10:00:00",
"timestamp_end": "2026-04-15T10:00:45",
"recorded_by": "PLC"
}
```

6.2 QC Checkpoint Record (law: x1030/x1040)

```
{
  "batch_id": "P260411-02-01-001",
  "checkpoint": "COOLDOWN",
  "phase_id": "x1030",

  "setpoint": {
    "temperature": 78.0,
    "temp_range": [75.0, 80.0],
    "brix_sp": "14.5",
    "ph_sp": "3.8"
  },

  "actual": {
    "temperature": 77.5,
    "brix": 14.3,
    "ph": 3.82,
    "ctw_temp": null,
    "steam_pressure": 2.1
  },

  "result": "PASS",
  "operator": "admin",
  "timestamp": "2026-04-15T10:15:30",
  "notes": ""
}
```

7. PLC Register Mapping Concept

Setpoint Registers (HMI → PLC) — Written per step

| Register | Type | Description |
|----------|------|--------------------|
| DB100.0 | INT | Step Number |
| DB100.2 | INT | Action Code |
| DB100.4 | REAL | Target Weight (kg) |
| DB100.8 | REAL | Temp Setpoint (°C) |
| DB100.12 | REAL | Temp Low Limit |
| DB100.16 | REAL | Temp High Limit |
| DB100.20 | REAL | Agitator RPM SP |
| DB100.24 | REAL | High Shear RPM SP |
| DB100.28 | INT | Step Time (s) |
| DB100.30 | REAL | Low Tolerance |
| DB100.34 | REAL | High Tolerance |

Actual/Telemetry Registers (PLC → HMI) — Read every 1s

| Register | Type | Description |
|----------|------|--|
| DB200.0 | INT | Current Step No |
| DB200.2 | INT | Step Timer (s) |
| DB200.4 | REAL | Mixing Tank Temp (°C) |
| DB200.8 | REAL | Mixing Tank Weight (kg) |
| DB200.12 | REAL | Agitator RPM Actual |
| DB200.16 | REAL | High Shear RPM Actual |
| DB200.20 | REAL | Hopper Weight (kg) |
| DB200.24 | REAL | Steam Pressure |
| DB200.28 | REAL | CTW Temp |
| DB200.32 | INT | Watchdog |
| DB200.34 | INT | Status (0=IDLE, 1=RUN, 2=PAUSE, 3=DONE, 9=ERROR) |

QC Registers (Operator Input or PLC Sensor)

| Register | Type | Description |
|----------|------|-------------------|
| DB300.0 | REAL | Actual Brix |
| DB300.4 | REAL | Actual pH |
| DB300.8 | REAL | Actual CTW Temp |
| DB300.12 | BOOL | QC Record Trigger |
| DB300.13 | BOOL | QC Pass/Fail |

8. Implementation Plan

Phase 1: Recipe Download (Done)

- ☒ Send ALL recipe to PLC via MQTT
- ☒ PLC Simulator handles recipe
- ☒ Download button + ACK

Phase 2: Step-by-Step Telemetry (Current)

- ☐ PLC simulator uses recipe SP for realistic telemetry
- ☐ Dashboard highlights active step with actual vs SP
- ☐ Step auto-advance based on step_time

Phase 3: Production Records

- ☐ Create production_records table in DB
- ☐ API endpoint to save per-step records
- ☐ Auto-save actual values when step completes

Phase 4: QC Checkpoints

- ☐ Detect QC steps (brix/ph/ctw flags)
- ☐ Show QC input dialog at x1030/x1040
- ☐ Compare actual vs SP → PASS/FAIL
- ☐ Save QC records to DB

Phase 5: Batch Report

- ☐ Generate batch production report
- ☐ Include all step records + QC results
- ☐ Export as PDF for QA sign-off

S7-1200 PLC Data Block & Interaction Design

Method: Step-by-Step Handshake (via Node-RED & MQTT)

This document defines the data structures (Data Blocks) used for real-time communication between the HMI/App and the Siemens S7-1200 PLC.

👉 DB100: Step Command (App → PLC)

Role: Receives recipe parameters and execution commands for the current step. **Access:** App writes (via MQTT/Bridge), PLC reads.

| Name | Data Type | Description | Example |
|------|-----------|-------------|---------|
|------|-----------|-------------|---------|

| | | | |
|----------------|----------------------|------------------------------------|---------------------|
| Watch_Doc | Int | Watchdog timer/counter from App | 1234 |
| Plan_ID | String[20] | Production Plan Identifier | 'PLAN-2024-001' |
| Batch_ID | String[20] | Current Production Batch | 'P260411-02-01' |
| SKU_Name | String[30] | Product name or SKU identifier | 'Strawberry_Jam_01' |
| Phase_ID | String[10] | Phase identifier code | 'A1010' |
| Step_ID | Int | Unique Step ID | 501 |
| Step_Time_SP | Int | Target Dwell/Mixing time (seconds) | 600 |
| Step_Status | Int | 0=Pending, 1=Active, 2=Complete | 1 |
| Material_ID | String[20] | Specific Raw Material ID | 'MAT-099' |
| Re_Code_ID | String[20] | Ingredient or Recipe Code | 'RM-SUG-01' |
| Req_Qty | Real | Required Quantity for step | 250.5 |
| TT_SP | Array[0..16] of Real | Temperature Profile Setpoints | [60.0, 65.0, ...] |
| Agitator_Speed | Real | Target Agitator Speed (RPM) | 30.0 |
| High_Shear_SP | Real | Target High Shear Speed (RPM) | 1500.0 |
| PH_Target | Real | Target pH level | 4.5 |
| Brix_Target | Real | Target Brix value | 12.5 |
| HMI_Command | Int | 0=IDLE, 1=START, 2=PAUSE, 3=ABORT | 1 |
| Cmd_NewStep | Bool | Trigger Flag to start the step | TRUE |

DB200: Telemetry (PLC → App)

Role: Live monitoring and feedback from the physical hardware. **Access:** PLC writes, App reads (polled or streamed every 1s).

| Name | Data Type | Description | Example |
|-----------------|-----------|---|---------|
| Watchdog | Int | Heartbeat signal from PLC | 32767 |
| PLC_State | Int | 0=Ready, 1=Run, 2=Hold, 9=Error | 1 |
| Current_Step | Int | Which step the PLC is actively executing | 5 |
| Step_Timer | Int | Elapsed time in current step (seconds) | 124 |
| Step_Status_Act | Int | Current Step Phase (e.g., 0=Init, 1=Dosing, 2=Mixing) | 2 |
| MixTank_Temp | Real | Live Tank Temperature (°C) | 59.5 |
| MixTank_Weight | Real | Live Tank Weight (kg) | 249.8 |
| Agitator_Act | Real | Live Agitator Speed (RPM) | 30.1 |
| HighShear_Act | Real | Live High Shear Speed (RPM) | 1498.2 |
| PH_Actual | Real | Live pH Sensor Reading | 4.52 |
| Brix_Actual | Real | Live Brix Sensor Reading | 12.4 |
| Hopper_Weight | Real | Live Sub-Hopper Weight (kg) | 0.0 |

Interaction Flow (Handshake)

The communication follows a strict handshake protocol to ensure safety in manual and automatic modes.

```text (Mermaid Diagram) sequenceDiagram participant App as Web Application participant NR as Node-RED / Bridge participant DB100 as PLC DB100 (CMD) participant DB200 as PLC DB200 (STATUS)

Note over App,DB200: Step Execution Flow

App->>NR: topic: plc/command (Step Data)  
NR->>DB100: Write All Params + Cmd\_NewStep = 1

Note over DB100: PLC Detects NewStep Trigger  
DB100->>DB100: Logic Starts Step  
DB100->>DB100: PLC Reset Cmd\_NewStep = 0 (ACK)

loop Every 1s  
    DB200->>NR: Current\_Step, Temp, Timer  
    NR->>App: UI Update (Live Dashboard)  
end

Note over DB200: PLC Finished Step  
DB200->>App: Step\_Status\_Act = Complete

### ## ✨ Implementation Guidance

- Watchdog Logic:** Both the App (DB100) and PLC (DB200) increment their watchdog values. If a side detects that the other side's value hasn't changed for >5 seconds, it should trigger a **Communication Error** and safe-stop any active motor/pump.
- String Handling:** S7 strings use the first two bytes for length metadata. The App serialization must account for this (2 bytes + actual string data).
- Trigger Reset:** The PLC should reset `Cmd_NewStep` to `FALSE` as soon as it acknowledges the new parameters, serving as a handshake confirmation back to the App.

---

### # DB1511 Memory Map (S7-1200 Non-Optimized)

This document maps the exact byte offsets for **DB1511 (DB\_FULL\_RECIPES)**. This is extremely important for writing your `python-snap7` `serialize()` function, as you must pack the binary payload to match these exact byte locations.

> **Important:** This assumes the DB is set to `S7_Optimized_Access := 'FALSE'` (Standard Access) so offsets are deterministic.

### ## 📦 1. `type\_FullRecipe` (Header & Array)

This is the top-level structure of DB1511. The header takes up the first 52 bytes, followed by the array of 128 steps.

| Offset | Name        | Data Type  | Size (Bytes) | Description                                                     |
|--------|-------------|------------|--------------|-----------------------------------------------------------------|
| 0      | Batch_ID    | String[20] | 22           | Batch Code (e.g. "P260411-02"). S7 strings have 2 header bytes. |
| 22     | SKU_ID      | String[20] | 22           | Recipe/SKU Code                                                 |
| 44     | HMI_Command | Int        | 2            | 0=IDLE, 1=START, 2=PAUSE, 3=ABORT, 9=RESET                      |
| 46     | Total_Steps | Int        | 2            | How many steps are actually populated (1-128)                   |

|                                                                                   |                  |                 |           |                                              |
|-----------------------------------------------------------------------------------|------------------|-----------------|-----------|----------------------------------------------|
| `+48.0`                                                                           | `Active_Step`    | Int             | 2         | PLC Pointer – current Seq number (1-128)     |
| `+50.0`                                                                           | `Cmd_LoadRecipe` | Bool            | 1 (bit 0) | Trigger bit to tell PLC a new array is ready |
| *padding*   *(Padding)*   *(Byte)*   1   S7 aligns Arrays to even byte boundaries |                  |                 |           |                                              |
| `+52.0`                                                                           | `Steps[1]`       | type_RecipeStep | 78        | First step (See Table 2)                     |
| `+130.0`                                                                          | `Steps[2]`       | type_RecipeStep | 78        | Second step                                  |
| `+208.0`                                                                          | `Steps[3]`       | type_RecipeStep | 78        | Third step                                   |
| ...                                                                               | ...              | ...             | 78        | ... repeats ...                              |
| `+9958.0`                                                                         | `Steps[128]`     | type_RecipeStep | 78        | Last step                                    |

\*(Total Size of DB1511: \*\*10,036 bytes\*\* ≈ 10 KB)\*

---

## ## 2. `type\_RecipeStep` (Individual Step Structure)

Each element in the `Steps[1..128]` array is exactly \*\*78 bytes\*\* long.

| Relative Offset | Name            | Data Type  | Size (Bytes) | Description                             |
|-----------------|-----------------|------------|--------------|-----------------------------------------|
| 0               | `Seq`           | Int        | 2            | Flat sequence number (1, 2, 3, ... 128) |
| 2               | `Phase_No`      | Int        | 2            | Phase number (10, 20, 30...)            |
| 4               | `Sub_Step`      | Int        | 2            | Step within phase (10, 20, 30...)       |
| 6               | `Action_Code`   | String[10] | 12           | Action code (e.g. "x10010")             |
| 18              | `Phase_ID`      | String[10] | 12           | Phase ID (e.g. "p0010")                 |
| 30              | `Re_Code`       | String[20] | 22           | Ingredient / Material code              |
| 52              | `Target_Weight` | Real       | 4            | Target dosing weight (kg)               |
| 56              | `Temp_SP`       | Real       | 4            | Temperature setpoint (°C)               |
| 60              | `Temp_Low`      | Real       | 4            | Min temperature limit (°C)              |
| 64              | `Temp_High`     | Real       | 4            | Max temperature limit (°C)              |
| 68              | `Agitator_SP`   | Real       | 4            | Agitator speed (RPM)                    |
| 72              | `HighShear_SP`  | Real       | 4            | High shear speed (RPM)                  |
| 76              | `Step_Time`     | Int        | 2            | Hold time (Seconds)                     |

\*(Total Size per Step: \*\*78 bytes\*\*)\*

---

## ## 3. Example: Cafe Amazon (S775743200) – Flattened into Array

This is how the database recipe for SKU "Cafe Amazon" would be flattened into the `Steps[1..128]` array:

| Seq    | Phase_No | Sub_Step | Action_Code | Phase_ID | Ingredient          | Target (kg) |
|--------|----------|----------|-------------|----------|---------------------|-------------|
| 1      | 10       | 10       | x10010      | p0010    | LS in Line          | 371.71      |
| 2      | 10       | 20       | x10020      | p0010    | R0-Water            | 372.87      |
| 3      | 10       | 30       | x10030      | p0010    | White Sugar W150    | 250.0       |
| 4      | 20       | 10       | x20010      | p0020    | *(empty)*           | 0.0         |
| 5      | 30       | 10       | x30010      | p0030    | Potassium Sorbate   | 0.8         |
| 6      | 30       | 20       | x30020      | p0030    | Sodium Benzoate     | 0.2         |
| 7      | 30       | 30       | x30030      | p0030    | Malic Acid          | 0.115       |
| 8      | 30       | 40       | x30040      | p0030    | Caramel Colour III  | 0.2         |
| 9      | 30       | 50       | x30050      | p0030    | R0-Water            | 4.0         |
| 10     | 40       | 10       | x40010      | p0040    | *(empty)*           | 0.0         |
| 11     | 45       | 10       | x45010      | p0045    | Sugar Flavour SG-01 | 0.11        |
| 12     | 50       | 10       | x50010      | p0050    | *(empty)*           | 0.0         |
| 13     | 60       | 10       | x60010      | p0060    | *(empty)*           | 0.0         |
| 14-128 | 0        | 0        |             |          | *(unused – Seq=0)*  | 0.0         |

> The PLC reads `Total\_Steps = 13`, so it knows to stop after Seq 13 and ignore Steps[14] to Steps[128].

```

Python Serialization Tip
When you write the `.serialize()` function in `plc_interface.py`:

```python
import struct

def pack_s7_string(s: str, max_len: int) -> bytes:
    s_bytes = s.encode('ascii')[:max_len]
    return struct.pack('BB', max_len, len(s_bytes)) + s_bytes.ljust(max_len, b'\x00')

# Pack Header (52 bytes)
header = b""
header += pack_s7_string(batch_id, 20)      # +0
header += pack_s7_string(sku_id, 20)        # +22
header += struct.pack('>h', hmi_command)     # +44
header += struct.pack('>h', total_steps)      # +46
header += struct.pack('>h', active_step)      # +48
header += struct.pack('?', cmd_load_recipe)  # +50
header += b'\x00'                           # +51 padding

# Pack Each Step (78 bytes each)
for step in steps:
    header += struct.pack('>hhh', step.seq, step.phase_no, step.sub_step) # +0,2,4
    header += pack_s7_string(step.action_code, 10) # +6
    header += pack_s7_string(step.phase_id, 10)    # +18
    header += pack_s7_string(step.re_code, 20)     # +30
    header += struct.pack('>ffffff',
        step.target_weight, step.temp_sp, step.temp_low,
        step.temp_high, step.agitator_sp, step.highshear_sp) # +52..+76
    header += struct.pack('>h', step.step_time)    # +76

# Total payload = 52 + (128 * 78) = 10,036 bytes
plc.db_write(db_number=1511, start=0, data=header)

```

TIA Portal SCL & DB Code for xMixing S7-1200

เอกสารชุดนี้คือ Source Code ในรูปแบบ SCL (Structured Control Language) สำหรับนำไปใช้ในโปรแกรม TIA Portal โดยตรง ประกอบด้วย Type Definitions (UDT), Data Blocks, และตัวอย่าง Function Block ในการควบคุม Handshake ขาเข้า-ออก

1. User Data Types (UDTs)

การกำหนด Type ทำให้โครงสร้างตัวแปรใน TIA Portal เป็นระเบียบและเรียกใช้งานซ้ำได้ง่าย

```

TYPE "type_StepCommand"
    TITLE = Step Command from Backend/HMI (v2)
    VERSION : 0.2
    STRUCT
        Watch_Doc      : Int;           // Watchdog counter
        Plan_ID         : String[20];   // รหัส Plan (เช่น P260411-02-01)
        Batch_ID        : String[20];   // รหัส Batch
        SKU_Name        : String[30];   // ชื่อ SKU
        Phase_ID        : String[10];   // รหัส Phase (เช่น A1010)

```

```

Step_No      : Int;           // ลำดับ Step ปัจจุบัน
Step_Time    : Int;           // เวลาทำงาน (วินาที)
Step_Status  : Int;           // 0=Pending, 1=Active, 2=Complete
Material_ID  : String[20];    // รหัสวัตถุดิบ SAP
Re_Code      : String[20];    // รหัสผสม/Ingredient
Target_Weight : Real;         // น้ำหนักที่ต้องการ (kg)
Temp_SP      : Real;          // Setpoint อุณหภูมิ (°C)
Temp_Low     : Real;          // Limit ต่ำสุด (°C)
Temp_High    : Real;          // Limit สูงสุด (°C)
Agitator_SP  : Real;          // ความเร็วใบกวน (RPM)
HighShear_SP : Real;          // ความเร็ว High Shear (RPM)
PH_Target    : Real;          // ค่า pH เป้าหมาย
Brix_Target  : Real;          // ค่า Brix เป้าหมาย
HMI_Command  : Int;           // 0=IDLE, 1=START, 2=PAUSE, 3=ABORT, 9=RESET
Cmd_NewStep  : Bool;          // Trigger bit จากระบบสั่งงานข้างนอก
END_STRUCT;
END_TYPE

```

TYPE "type_RecipeStep"

TITLE = Individual Step Definition (**For Array**)

VERSION : 0.2

```

STRUCT
Seq      : Int;           // ลำดับที่ในอาร์เรย์ (1, 2, 3, ... 128)
Phase_No : Int;           // เลข Phase เช่น 10, 20, 30
Sub_Step  : Int;          // เลข Step ภายใน Phase เช่น 10, 20, 30
Action_Code : String[10]; // รหัส Action (เช่น x10010, x20010)
Phase_ID  : String[10];   // รหัส Phase (เช่น p0010, p0020)
Re_Code   : String[20];   // รหัสผสม/Ingredient
Target_Weight : Real;     // น้ำหนักที่ต้องการ (kg)
Temp_SP   : Real;         // Setpoint อุณหภูมิ (°C)
Temp_Low  : Real;         // Limit ต่ำสุด
Temp_High : Real;         // Limit สูงสุด
Agitator_SP : Real;       // ความเร็วใบกวน (RPM)
HighShear_SP : Real;      // ความเร็ว High Shear (RPM)
Step_Time : Int;          // เวลาทำงาน (วินาที)
END_STRUCT;
END_TYPE

```

// ตัวอย่าง Flat Sequence:

```

// Seq=1 Phase_No=10 Sub_Step=10 Action_Code="x10010" (P10 Step 10)
// Seq=2 Phase_No=10 Sub_Step=20 Action_Code="x10020" (P10 Step 20)
// Seq=3 Phase_No=20 Sub_Step=10 Action_Code="x20010" (P20 Step 10)
// Seq=4 Phase_No=20 Sub_Step=20 Action_Code="x20020" (P20 Step 20)
// Seq=5 Phase_No=20 Sub_Step=30 Action_Code="x20030" (P20 Step 30)
// Seq=6 Phase_No=20 Sub_Step=40 Action_Code="x20040" (P20 Step 40)

```

TYPE "type_FullRecipe"

TITLE = Full Recipe Command (All Steps) from Node-RED/HMI

VERSION : 0.2

```

STRUCT
Batch_ID      : String[20]; // รหัส Batch
SKU_ID        : String[20]; // รหัส SKU
HMI_Command   : Int;        // 0=IDLE, 1=START, 2=PAUSE, 3=ABORT, 9=RESET
Total_Steps   : Int;        // จำนวน Step ทั้งหมดใน Recipe นี้ (1-128)
Active_Step   : Int;        // Pointer ซึ่งขั้นตอนที่ต้องทำ Seq ใด (1-128)
Cmd_LoadRecipe : Bool;      // Trigger โหลด Recipe ใหม่
Steps         : Array[1..128] of "type_RecipeStep"; // รองรับสูงสุด 128 Steps
END_STRUCT;
END_TYPE

```

TYPE "type_Telemetry"

TITLE = Live Data to Node-RED/HMI

```

VERSION : 0.1
STRUCT
    Watchdog      : Int;           // สัญญาณชีพ
    PLC_State      : Int;           // 0=Ready, 1=Run, 2=Hold, 3=Pause, 9=Error
    Current_Step   : Int;           // Step ที่ PLC กำลังทำงาน
    Step_Timer     : Int;           // จัณเวลาผ่านไปแล้วกี่วินาที
    MixTank_Temp   : Real;          // อุณหภูมิในถัง
    MixTank_Weight : Real;          // น้ำหนักในถัง
    Agitator_Act   : Real;          // ความเร็วจริงใบกวน
    HighShear_Act  : Real;          // ความเร็วจริง High Shear
    Hopper_Weight  : Real;          // น้ำหนักใน Hopper ย่อย
END_STRUCT;
END_TYPE

TYPE "type_Handshake"
TITLE = End Step Response
VERSION : 0.1
STRUCT
    Step_Complete : Bool;          // Pulse แจ้งว่า Step จบแล้วจ้า
    Finished_Step : Int;           // ยืนยันจบ Step เบอร์...
    End_Temp       : Real;          // Snapshot อุณหภูมิ
    End_Weight     : Real;          // Snapshot น้ำหนัก
    Error_Flag     : Bool;          // ถ้ามีการหยุดจาก Alarm
    Error_Code     : Int;           // Error รหัสไหน
END_STRUCT;
END_TYPE

```

2. Data Blocks (DBs)

กำหนด Global DB โดยเรียกใช้ UDT ด้านบน

```

DATA_BLOCK "DB_STEP_CMD"
TITLE = DB1510 : Command From HMI
{ S7_Optimized_Access := 'FALSE' } // ตั้งเป็น Standard access (เพื่อ Node-RED ดึงแบบ Absolute Ad
AUTHOR : 'AW'
FAMILY : 'MIX'
VERSION : 0.1
NON_RETAIN
    "type_StepCommand"
BEGIN
END_DATA_BLOCK

DATA_BLOCK "DB_FULL_RECIPE"
TITLE = DB1511 : Full Recipe Array From HMI
{ S7_Optimized_Access := 'FALSE' }
AUTHOR : 'AW'
FAMILY : 'MIX'
VERSION : 0.1
NON_RETAIN
    "type_FullRecipe"
BEGIN
END_DATA_BLOCK

DATA_BLOCK "DB_TELEMETRY"
TITLE = DB1512 : Telemetry To HMI
{ S7_Optimized_Access := 'FALSE' }
AUTHOR : 'AW'
FAMILY : 'MIX'
VERSION : 0.1

```

```
NON_RETAIN
    "type_Telemetry"
BEGIN
END_DATA_BLOCK

DATA_BLOCK "DB_HANDSHAKE"
TITLE = DB1513 : Step Confirmation To HMI
{ S7_Optimized_Access := 'FALSE' }
AUTHOR : 'AW'
FAMILY : 'MIX'
VERSION : 0.1
NON_RETAIN
    "type_Handshake"
BEGIN
END_DATA_BLOCK
```

3. Handshake Function Block (FB)

FB หลักที่ทำหน้าที่ประมวลผลสัญญาณรับเข้า และเคลียร์ flag ต่างๆ อย่างปลอดภัย (Safe Pulse/Latch Logic)

```
FUNCTION_BLOCK "FB_Step_Handshake"
TITLE = Step Flow Controller
VERSION : 0.1
VAR_INPUT
    Reset_All    : Bool;    // กดตอนจะเริ่ม Batch ใหม่หรือล้าง Error
END_VAR

VAR
    ton_StepDelay : TON_TIME; // หน่วงเวลาก่อนตัดจบสแต็ป
    tp_CompletePulse : TP_TIME; // Pulse ออกไปให้ Node-RED (1 s)
    Internal_State : Int;      // State Machine ของ FB
                                // (0=Idle, 1=Loading, 2=Running, 3=Complete)
END_VAR

BEGIN
    // -----
    // 1. รับ Trigger จาก HMI ว่ามี Step ใหม่เข้ามา
    // -----
    IF "DB_STEP_CMD".Cmd_NewStep AND (#Internal_State = 0 OR #Internal_State = 3) THEN
        // นำข้อมูลเข้าสู่สถานะเริ่มทำงาน
        "DB_TELEMETRY".Current_Step := "DB_STEP_CMD".Step_No;
        "DB_TELEMETRY".Step_Timer := 0;
        "DB_TELEMETRY".PLC_State := 1; // Running

        // ปิด Flag รับทราบ ให้ Node-RED รู้ว่ารับแล้ว
        "DB_STEP_CMD".Cmd_NewStep := FALSE;

        // อัปเดตสถานะในเครื่อง
        #Internal_State := 2; // ย้ายไปสถานะทำงานเครื่องขุมพลัง
    END_IF;

    // -----
    // 2. จำลองการประมวลผล (หรือทำงานร่วมกับ PID / Timer)
    // -----
    IF #Internal_State = 2 THEN

        // >>>> ตรงนี้คือลอจิกควบคุมเครื่องของจริงที่จะอ่าน Temp_SP, RPM เอาไปคุม Inverter <<<<
```



```

// ตรวจสอบเงื่อนไขเวลา
IF "DB_STEP_CMD".Step_Time > 0 AND "DB_TELEMETRY".Step_Timer >= "DB_STEP_CMD".Step_T
THEN

    #Internal_State := 3; // เวลาหมด แปลว่า Step สมบูรณ์
END_IF;
END_IF;

// -----
// 3. ปิดจบและส่ง Status ให้ Node-RED
// -----
IF #Internal_State = 3 THEN
    // 3.1 บันทึกค่า Snapshot ตอนจบ
    "DB_HANDSHAKE".Finished_Step := "DB_TELEMETRY".Current_Step;
    "DB_HANDSHAKE".End_Temp      := "DB_TELEMETRY".MixTank_Temp;
    "DB_HANDSHAKE".End_Weight    := "DB_TELEMETRY".MixTank_Weight;

    // 3.2 สร้าง Pulse (Pulse 1 วนาก้ ให้ Node-RED แปลงว่าอ่านกัน)
    #tp_CompletePulse(IN := TRUE, PT := T#1s);
    "DB_HANDSHAKE".Step_Complete := #tp_CompletePulse.Q;

    // 3.3 เมื่อจบ Pulse แล้ว เปลี่ยนกลับไปรอคำสั่งถัดไป
    IF NOT #tp_CompletePulse.Q THEN
        #tp_CompletePulse(IN := FALSE, PT := T#1s); // เคลียร์ Time
        #Internal_State := 0; // กลับไปช่อง IDLE รอรับ Cmd_NewStep อีกครั้ง
        "DB_TELEMETRY".PLC_State := 0; // Ready
    END_IF;
END_IF;

// -----
// X. Reset ทั้งระบบ
// -----
IF #Reset_All OR "DB_STEP_CMD".HMI_Command = 3 THEN // HMI_Command 3 = Abort
    #Internal_State := 0;
    "DB_TELEMETRY".PLC_State := 0;
    "DB_STEP_CMD".Cmd_NewStep := FALSE;
    "DB_HANDSHAKE".Step_Complete := FALSE;
    #tp_CompletePulse(IN := FALSE, PT := T#1s);
END_IF;

END_FUNCTION_BLOCK

```

สรุปการตั้งค่าบน TIA Portal:

- สร้าง UDT 3 ตัวก่อน
- สร้าง Data Block (DB1510, DB1512, DB1513) และกลุ่ม UDT
- ข้อควรระวังสำคัญ:** ถ้าใช้ Node-RED อ่านผ่าน s7-comm หรือโปรโตคอล S7 ควรกดคลิกขวาที่ Data block -> Properties -> เอาสัญลักษณ์ติดถูกหน้า Optimized block access ออก เพื่อให้ Address มีค่าพิกัดแน่นอน (เช่น DB1510.DBW0) Node-RED จะเห็นค่าถูกต้อง
- ให้ฟังก์ชันเพิ่มใน OB1 หรือ Time Interrupt OB (ทุก 1 วินาที) วนคอยบวก DB_TELEMETRY.Watchdog และ DB_TELEMETRY.Step_Timer

📊 SKU Recipe Analysis — All 16 Active SKUs

1. Full SKU Breakdown

SKU ID	Product Name	Steps	Phases	Max Steps/Phase	Avg Steps/Phase
S77S743200	Cafe Amazon	13	7	5	1.9
SFMU5USN00	Fresh Mint Senorita 750ml	15	8	4	1.9
S7GCA4SNA0	Japanese Melon Senorita 1900ml	17	8	8	2.1
S7GU5USN00	Japanese Melon Senorita 750ml	17	8	8	2.1
S7KCY4SNA0	Coconut Senorita 1900ml	17	9	4	1.9
S7KU5USN00	Coconut Senorita 750ml	17	9	4	1.9
S77CY4SN00	Classic Caramel Senorita	18	9	6	2.0
SFLDBUSS00	Rose Senorita Signature 720ml	24	10	7	2.4
S7EFRU4200- 1	Strawberry Freshy 710ml x12	30	15	5	2.0
S7EFSU4200	Strawberry Freshy 710ml x3	30	15	5	2.0
S7HFRU4200	Orange Freshy 710ml x12	31	12	15	2.6
S7CFSU4200	Blue Lemon Freshy 710ml x3	32	17	5	1.9
S7DFRU4200	Lychee Freshy 710ml x12	34	16	5	2.1
S7DFSU4200	Lychee Freshy 710ml x3	34	16	5	2.1
S7YFRU4200	Green Apple Freshy 710ml x12	34	18	5	1.9
SFGFSU4200	Blue Hawaii Freshy 710ml x3	35	17	5	2.1

2. Statistical Summary

16 SKUs Analyzed			
Metric	Min	Max	Avg
Steps per SKU	13	35	24.9
Phases per SKU	7	18	12.1
Max steps in 1 phase	-	15	-
Avg steps per phase	1.9	2.6	2.1

3. Phase Type Classification

พบ 3 ประเภท Phase ในทุก SKU:

Prefix	Type	หน้าที่	ตัวอย่าง
A	Auto Batching	ซึ่งตรวจวัดวัตถุดิบอัตโนมัติ (น้ำ, น้ำตาล, LS)	A1010, A1020
D	Dissolve / Mix	ละลาย, ผสม, ต้ม, เติมวัตถุดิบ	D1010, D1030
x	Transfer / Control	โอนย้ายถัง, รอ, QC, จบกระบวนการ	x1010, x1020, x1030, x1040

Process Flow Pattern (พบในทุก SKU):

text (Mermaid Diagram) graph LR A[A - Auto Batching
ซึ่งตรวจวัดวัตถุดิบอัตโนมัติ] --> X1[x - Transfer
โอนเข้าถังผสม] X1 --> D[D - Dissolve
ละลาย/ผสม] D --> X2[x - Transfer
โอนออก] X2 --> X3[x1020 - QC Check] X3 --> X4[x1030 - Storage] X4 --> X5[x1040 - Complete]

4. Payload Size Analysis

Method	Worst Case	Typical	Comment
Send ALL	35 steps × 200B = 6.8 KB	25 × 200B = 4.9 KB	เล็กมาก!
Send per Phase	15 steps × 200B = 2.9 KB	2 × 200B = 0.4 KB	เล็กเกินไป
Send per Step	1 × 200B = 0.2 KB	-	ไม่คุ้ม overhead

[!!IMPORTANT] ข้อมูลสำคัญ: Payload ของสูตรที่ใหญ่ที่สุด (35 steps) มีขนาดเพียง **6.8 KB** เท่านั้น — เล็กมากๆ สำหรับ PLC สมัยใหม่ที่มี memory เป็น MB ขึ้นไป

5. 🛡️ คำแนะนำ: Send ALL (Option A)

เหตุผล:

#	เหตุผล	รายละเอียด
1	Payload เล็กมาก	แม้ SKU ที่ซับซ้อนสุด (35 steps) ก็แค่ 6.8 KB — PLC รับได้สบายๆ
2	Avg 2.1 steps/phase	ถ้าส่งทีละ Phase จะต้อง handshake 12-18 ครั้ง เปลืองเกินไป
3	PLC ทำงานอิสระ	ไม่ต้องรอ Network ระหว่างขั้นตอน — ปลอดภัยกว่า สำหรับโรงงาน
4	ง่ายต่อการ Debug	ส่งครั้งเดียว ตรวจสอบว่าครบถ้วน แล้ว PLC รันตัวเอง
5	Network Failure Safe	ถ้าเน็ต MQTT หลุดระหว่างผลิต PLC ยังวิ่งต่อได้

Architecture ที่แนะนำ:

```text (Mermaid Diagram) sequenceDiagram participant OP as Operator participant HMI as x31 Dashboard participant MQTT as MQTT Broker participant PLC as PLC

OP->>HMI: Click "Start Production"

HMI->>HMI: Fetch all SKU steps from DB

HMI->>MQTT: mixing/plant/1/recipe<br>{batch\_id, sku\_id, batch\_size,<br>steps: [all 13-35 steps]}

MQTT->>PLC: Full recipe (max 6.8 KB)

PLC->>MQTT: mixing/plant/1/ack<br>{status: "RECIPE\_LOADED", steps: 35}

Note over HMI: Recipe confirmed, PLC ready

OP->>HMI: Click "START"

HMI->>MQTT: mixing/plant/1/cmd<br>{command: "START"}

loop PLC executes autonomously

PLC->>MQTT: mixing/plant/1/telemetry<br>{step\_no, temp, rpm, weight, watchdog}

HMI->>HMI: Update gauges in real-time

end

PLC->>MQTT: mixing/plant/1/status<br>{status: "BATCH\_COMPLETE"}

### MQTT Topic Design (Final):

mixing/plant/{id}/ |—— recipe # HMI → PLC: Full recipe download (ALL steps) |—— cmd # HMI → PLC: START, PAUSE, RESUME, ABORT |—— ack # PLC → HMI: Recipe loaded / Command acknowledged |—— telemetry # PLC → HMI: Real-time sensor data (every 1-2s) |—— status # PLC → HMI: RUNNING, PAUSED, COMPLETE, ERROR |—— watchdog # PLC → HMI: Heartbeat counter

### Recipe JSON Schema:

```
```json
{
  "batch_id": "P260411-02-01-001",
  "sku_id": "S77S743200",
  "sku_name": "Cafe Amazon",
  "batch_size": 1200.0,
  "plant_id": 1,
  "total_steps": 13,
  "total_phases": 7,
  "timestamp": "2026-04-15T16:40:00",
  "steps": [
    {
      "step_no": 10,
      "phase": "p0010",
      "phase_id": "A1010",
      "action_code": "10030",
      "re_code": "LS in Line",
      "destination": "1010",
      "require": 371.71,
      "uom": "kg",
      "low_tol": 0.001,
      "high_tol": 0.001,
      "agitator_rpm": 80.0,
      "high_shear_rpm": 0.0,
      "temperature": 40.0,
      "temp_low": 0.0,
    }
  ]
}
```

```
    "temp_high": 0.0,  
    "step_time": 0,  
    "step_timer_control": 0  
  }  
]  
}
```

6. Next Steps

- ☐ Implement recipe publish in `x61-MixingControl.vue` (Send All on Start)
 - ☐ Update `plc_simulator.mjs` to receive & parse full recipe
 - ☐ Add recipe ACK handling in `useMQTT.ts`
 - ☐ Add step progress tracking on dashboard (current step highlight)
-

Large Recipe Transfer Protocol Concept

Purpose: To explain the step-by-step S7 logic for safely transferring massive SKUs (e.g., up to 32 Phases) from the App to the PLC without hitting hardware buffer limits.

1. The Hardware Limitation

The `DB1780_RecipeData` datablock is very large (about **29 KB**). However, the Siemens S7-1200 hardware protocol has a “PDU Size Limit” (Protocol Data Unit), meaning it can physically only accept about **240 bytes of data per single write request** at the lowest level.

If we attempted to send all 32 Phases in one single JSON command to `node-red-contrib-s7`, the node would buffer overflow and crash.

2. The Solution: Dynamic Chunking

By breaking the recipe down so that the Node-RED logic writes “**One Step at a time**”, the network traffic stays perfectly stable.

A single `UDT_ProcessStep` is about 100 bytes, which perfectly fits inside a single, fast S7 packet. With a ~15-20ms delay between writes, a massive 250-step recipe takes exactly **4 to 5 seconds** to transfer safely with zero data loss.

3. The Step-by-Step Transfer Queue (Node-RED logic)

Imagine FastAPI sends a huge recipe with **32 Phases** to Node-RED. Node-RED will build an array of write commands (a queue) and send them sequentially to the S7 node.

Stage 1: PREPARE (Lock the PLC)

Node-RED sends the first commands to secure the machine, ensuring no operator can start the sequence while data is downloading.

```
// Send command 1 & 2
Write -> Variable: "DB1780,X120.0" (RecipeLoading) Value: TRUE
Write -> Variable: "DB1780,X121.0" (RecipeReady) Value: FALSE
```

Stage 2: TRANSFER HEADER (Basic Info)

Node-RED writes the basic string and float data for the batch header.

```
// Send commands 3 through 10
Write -> Variable: "DB1780,S0.30" (PlanID) Value: "P260309-01-01"
Write -> Variable: "DB1780,S32.20" (BatchID) Value: "P260309-01-01-001"
Write -> Variable: "DB1780,S54.20" (SkuID) Value: "SFGFSU4200"
Write -> Variable: "DB1780,REAL110" (BatchSize) Value: 4200.0
Write -> Variable: "DB1780,INT114" (ProcessCount) Value: 32 <-- Max phases!
Write -> Variable: "DB1780,DINT122" (CRC_App) Value: 3408960
```

Stage 3: TRANSFER PROCESSES (The Big Loop)

Because this is the biggest SKU, Node-RED runs a for loop from Phase 0 to Phase 31. Instead of writing every tiny integer one-by-one (which would take 5,000 writes), Node-RED writes a **whole Step at a time**.

```
// ---- Phase 0 (Process 10) ----
// Write Process Metadata
Write -> Variable: "DB1780,INT356" (Processes[0].ProcessNo) Value: 10
Write -> Variable: "DB1780,INT358" (Processes[0].PhaseID) Value: 401
Write -> Variable: "DB1780,INT360" (Processes[0].StepCount) Value: 4

// Write Step 0
Write -> Variable: "DB1780,INT364" (Processes[0].Steps[0].StepNo) Value: 10
Write -> Variable: "DB1780,INT366" (Processes[0].Steps[0].ActionCode) Value: 10010
Write -> Variable: "DB1780,S368.25" (Processes[0].Steps[0].ReCode) Value: "R0-Water"
Write -> Variable: "DB1780,REAL418" (Processes[0].Steps[0].Require) Value: 1200.0
Write -> Variable: "DB1780,REAL430" (Processes[0].Steps[0].Temperature) Value: 85.0
// ... NodeRED continues for all properties of Step 0

// Write Step 1
Write -> Variable: "DB1780,INT464" (Processes[0].Steps[1].StepNo) Value: 20
Write -> Variable: "DB1780,INT466" (Processes[0].Steps[1].ActionCode) Value: 21010
// ... NodeRED continues for all properties of Step 1

// ---- Phase 1 (Process 20) ----
Write -> Variable: "DB1780,INT1156" (Processes[1].ProcessNo) Value: 20
// ... Writes all steps for Phase 1 ...

// ... (Loops all the way down to Phase 31) ...
```

Stage 4: VERIFY AND ACTIVATE

Once the loop finishes (after about 3-4 seconds of rapid-fire S7 messages), Node-RED unlocks the PLC.

Because we sent it in over 150 pieces, what happens if the Wi-Fi drops and a piece gets lost? Node-RED relies on the CRC_App variable sent in Stage 2.

```
// Release the lock
```

```
Write -> Variable: "DB1780,X120.0" (RecipeLoading) Value: FALSE

// The PLC now automatically runs FC_CalcChecksum inside its own processor.
// It calculates the exact sum of all 250 steps it received.
// Node-RED polls the status to check if it worked:
Read  <- Variable: "DB1780,X126.0" (CRC_Match)      Result: TRUE!

// Since it matched perfectly, Node-RED gives the final green light
Write -> Variable: "DB1780,X121.0" (RecipeReady)   Value: TRUE
```

The UI turns green, and the operator is now allowed to press START. We are 100% certain that no data was dropped.

Recipe Transfer Protocol — App to PLC

How to reliably transfer ~29 KB of recipe data to the PLC
and **prove** every byte arrived correctly.

The Challenge

Issue	Detail
PDU Size Limit	S7-1200 has a max PDU (Protocol Data Unit) of 240 bytes per request . You CANNOT write 29 KB in one shot.
Network Reliability	Ethernet packets can be dropped, delayed, or corrupted
Partial Write	If the transfer is interrupted halfway, the PLC has a half-loaded recipe = DANGEROUS
Verification	How does the operator know ALL 13 steps arrived correctly?

Solution: Chunked Transfer with Checksum Verification

The 4-Stage Protocol

Stage 1 PREPARE	Stage 2 TRANSFER	Stage 3 VERIFY	Stage 4 ACTIVATE
Lock recipe Set flag RecipeReady = FALSE	Write chunks phase by phase via Node-RED S7 writes	Read back & compare CRC App vs PLC checksum match?	Set RecipeReady = TRUE PLC can start

Stage 1: PREPARE — Lock the Recipe Slot

Before writing any data, tell the PLC “I’m about to download a new recipe.”

App → Node-RED → PLC:

DB1780.RecipeReady	= FALSE	← PLC must NOT start with old data
DB1780.RecipeLoading	= TRUE	← New flag: download in progress
DB1780.BatchID	= ""	← Clear old batch
DB1780.CRC_App	= 0	← Clear old checksum

PLC Logic: If RecipeLoading = TRUE, ignore any StartCmd. This prevents accidental start during transfer.

Stage 2: TRANSFER — Write Data in Chunks

How S7 Protocol Works

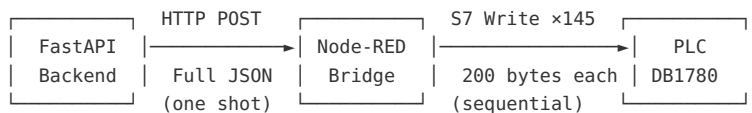
One S7 Write Request = max ~200 bytes of data (within 240-byte PDU)

DB1780 total = ~29,000 bytes

29,000 ÷ 200 = 145 write requests needed (takes ~2-3 seconds)

Node-RED Handles the Chunking

The App does NOT write directly to the PLC. Instead:



Node-RED Flow:

```
// Node-RED Function: Break recipe JSON into S7 write commands
const recipe = msg.payload;

// 1. Write Header fields one by one
const writes = [];
writes.push({ addr: "DB1780,S0.30", value: recipe.Header.PlanID });
writes.push({ addr: "DB1780,S32.20", value: recipe.Header.BatchID });
writes.push({ addr: "DB1780,S54.20", value: recipe.Header.SkuID });
// ... etc

// 2. Write each Process/Step
for (let p = 0; p < recipe.Processes.length; p++) {
  const proc = recipe.Processes[p];
  const baseOffset = HEADER_SIZE + (p * PROCESS_SIZE);

  // Write process header
  writes.push({ addr: `DB1780,INT${baseOffset}`, value: proc.ProcessNo });
  writes.push({ addr: `DB1780,INT${baseOffset+2}`, value: proc.PhaseID });

  // Write each step (8 per process)
  for (let s = 0; s < 8; s++) {
    const step = proc.Steps[s];
    const stepOffset = baseOffset + PROC_HEADER + (s * STEP_SIZE);

    writes.push({ addr: `DB1780,INT${stepOffset}`, value: step.StepNo });
    writes.push({ addr: `DB1780,INT${stepOffset+2}`, value: step.ActionCode });
    writes.push({ addr: `DB1780,REAL${stepOffset+4}`, value: step.Require });
    // ... etc for all fields
  }
}
```



```

}

// 3. Execute writes sequentially
msg.writes = writes;
return msg;

```

Transfer Time Estimate

PLC Model S7 Write Speed Time for 145 writes

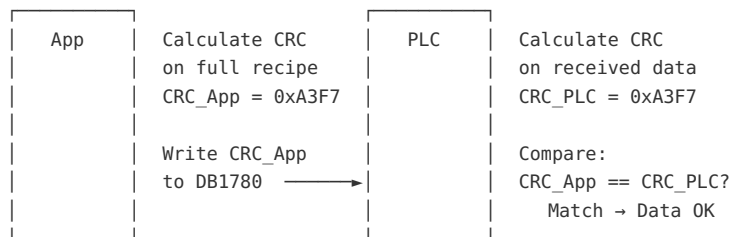
S7-1200 ~10-15ms per write **~1.5-2.2 seconds**

S7-1500 ~5-8ms per write **~0.7-1.2 seconds**

This is fast enough — operator won't even notice.

Stage 3: VERIFY — Prove No Data Lost

Method A: CRC Checksum (Recommended)



App Side (Python/FastAPI):

```

import struct
import zlib

def calculate_recipe_crc(recipe_data: dict) -> int:
    """Calculate CRC32 of all recipe values in order."""
    buffer = bytearray()

    # Pack header
    buffer += recipe_data["Header"]["BatchID"].encode().ljust(20, b'\x00')
    buffer += recipe_data["Header"]["SkuID"].encode().ljust(20, b'\x00')
    buffer += struct.pack('>H', recipe_data["Header"]["ProcessCount"])

    # Pack each step in order
    for proc in recipe_data["Processes"]:
        buffer += struct.pack('>H', proc["ProcessNo"])
        buffer += struct.pack('>H', proc["StepCount"])
        for step in proc["Steps"]:
            buffer += struct.pack('>H', step["StepNo"])
            buffer += struct.pack('>H', step["ActionCode"])
            buffer += struct.pack('>f', step["Require"])
            buffer += struct.pack('>f', step["Temperature"])
            buffer += struct.pack('>f', step["AgitatorRPM"])
            buffer += struct.pack('>f', step["HighShearRPM"])
            buffer += struct.pack('>H', step["StepTime"])

```

```

    crc = zlib.crc32(buffer) & 0xFFFFFFFF
    return crc

```

PLC Side (SCL):

```

// Simple checksum: sum all critical numeric values
// (Full CRC32 is complex in SCL, use additive checksum instead)

FUNCTION "FC_CalcChecksum" : DInt
VAR_TEMP
    i, j      : Int;
    checksum  : DInt;
    stepData  : "UDT_ProcessStep";
END_VAR

BEGIN
    #checksum := 0;

    FOR #i := 0 TO "DB_RecipeData".ProcessCount - 1 DO
        #checksum := #checksum + "DB_RecipeData".Processes[#i].ProcessNo;
        #checksum := #checksum + "DB_RecipeData".Processes[#i].StepCount;

        FOR #j := 0 TO "DB_RecipeData".Processes[#i].StepCount - 1 DO
            #stepData := "DB_RecipeData".Processes[#i].Steps[#j];
            #checksum := #checksum + #stepData.StepNo;
            #checksum := #checksum + #stepData.ActionCode;
            #checksum := #checksum + REAL_TO_DINT(#stepData.Require * 100.0);
            #checksum := #checksum + REAL_TO_DINT(#stepData.Temperature * 100.0);
            #checksum := #checksum + REAL_TO_DINT(#stepData.AgitatorRPM);
            #checksum := #checksum + REAL_TO_DINT(#stepData.HighShearRPM);
            #checksum := #checksum + #stepData.StepTime;
        END_FOR;
    END_FOR;

    "FC_CalcChecksum" := #checksum;
END_FUNCTION

```

Method B: Readback Verification (Simpler but Slower)

After writing all data, Node-RED reads back every field and the App compares:

App sends recipe → Node-RED writes to PLC → Node-RED reads back from PLC → App compares

For each field:

```

IF sent_value != readback_value THEN
    → ERROR: Data mismatch at Process[2].Steps[3].Temperature
    → RETRY transfer

```

Pros: Simple, no PLC code needed

Cons: Takes 2× time (write + read), ~4 seconds total

Method C: Step Count + Key Field Spot-Check (Fastest)

Quick verification — don't check everything, just critical fields:

App writes:

```

ProcessCount = 7
TotalSteps = 13
Processes[0].Steps[0].ActionCode = 10030

```

Node-RED reads back these 5 values and compares.
If all match → 99.99% confidence data is correct.

Only after verification passes:

```
DB1780.CRC_App      = calculated_checksum
DB1780.RecipeLoading = FALSE
DB1780.RecipeReady  = TRUE      ← NOW the PLC is allowed to start
```

```
// Only allow start if recipe is verified
IF "DB_RecipeData".StartCmd AND "DB_RecipeData".RecipeReady
AND NOT "DB_RecipeData".RecipeLoading THEN

    // Optional: PLC calculates its own checksum and compares
    IF "FC_CalcChecksum"() = "DB_RecipeData".CRC_App THEN
        "DB_RecipeData".PLC_State := 1; // Start!
    ELSE
        "DB_RecipeData".Error_Code := 4; // CRC mismatch!
        // DO NOT START – recipe may be corrupted
    END_IF;
END_IF;
```

```

📄 Downloading Recipe to PLC...

██████████████████████████████████████░░░░░░░░░░░░░░░░░░░░░░ 78% |

Phase 1/7: p0010 – Batching .....
Phase 2/7: p0020 – Heating .....
Phase 3/7: p0030 – Dissolving .....
Phase 4/7: p0040 – Pre-Heat .....
Phase 5/7: p0045 – Flavorize .....
Phase 6/7: p0050 – Pasteurize .....
Phase 7/7: p0060 – Cooling .....

Verifying checksum...

```

```
Recipe Downloaded Successfully!

Batch: P260309-01-01-001
SKU: S77S743200 - Cafe Amazon
Steps: 13 steps in 7 phases
CRC: 0xA3F7B2C1 Match
```



Recommended Approach

Method	Reliability	Speed	PLC Code Needed?
A: CRC Checksum	☆☆☆☆☆	Fast (2s)	Yes (FC_CalcChecksum)
B: Full Readback	☆☆☆☆☆	Slow (4s)	No
C: Spot-Check	☆☆☆☆	Fastest (1s)	No

My Recommendation: Use **Method A (CRC) + Method C (Spot-Check)** together: 1. Calculate additive checksum on App side 2. Write recipe + checksum to PLC 3. Quick spot-check: read back ProcessCount + first step + last step 4. PLC calculates its own checksum on StartCmd 5. If both match → production is safe to start

New DB1780 Fields Required

Add these to the header section:

```
// — Transfer Control —
RecipeLoading : Bool; // TRUE during transfer (blocks StartCmd)
CRC_App : DInt; // Checksum calculated by App
CRC_PLC : DInt; // Checksum calculated by PLC
CRC_Match : Bool; // TRUE if CRC_App == CRC_PLC
Transfer_Error : Int; // 0=OK, 1=Timeout, 2=CRC Mismatch
```

Failure Recovery

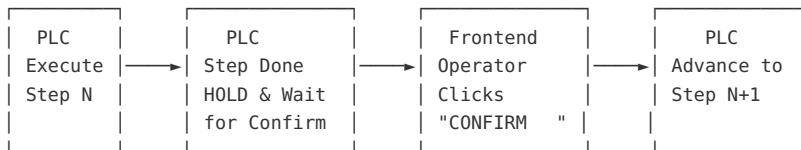
Scenario	What Happens
Network drops during transfer	RecipeLoading = TRUE, RecipeReady = FALSE → PLC cannot start. App retries from scratch.
CRC mismatch	App shows error “Recipe verification failed!” + retries automatically (up to 3 times)
PLC power loss during transfer	DB1780 is in retentive memory. On power-up, RecipeLoading = TRUE → PLC stays idle. App must re-download.
App crashes during transfer	Same as network drop — PLC stays in “loading” state. App restart detects RecipeLoading = TRUE and re-downloads.

Step Confirmation Concept — Operator Must Confirm Every Step

The Problem

In the current “full auto” design, the PLC auto-advances after timer/condition is met. But in real food manufacturing, the **operator must visually verify** each step is actually done before moving forward (e.g., “Is the sugar fully dissolved?”, “Did the temperature reach 83°C?”).

The New Workflow: “Execute → Hold → Confirm → Advance”



PLC State Machine (Updated)

State 0: IDLE	→ Waiting for recipe download + Start command
State 1: EXECUTING	→ Running current step (timer counting, outputs ON)
State 2: WAIT_CONFIRM	→ Step finished, holding outputs, waiting for operator
State 3: PAUSED	→ Operator paused the batch
State 4: DONE	→ All steps complete
State 9: ERROR	→ Fault condition

Key Change: After every step completes, PLC goes to **State 2 (WAIT_CONFIRM)** instead of directly advancing. The agitator/temperature HOLD at current setpoint while waiting, so the product is safe.

Data Block Updates (DB1780)

Add these fields to the header:

```
// — Confirmation Handshake —
Step_Done          : Bool;    // PLC sets TRUE when step execution is finished
Confirm_Step       : Bool;    // PC sets TRUE to acknowledge and advance
Confirm_Phase_ID   : String[10]; // PC echoes back which phase it's confirming
Confirm_Step_ID    : Int;     // PC echoes back which step it's confirming
```

PLC Logic Change (in FB1780):

```
// After step timer/condition is met:
IF step_finished THEN
    "DB_RecipeData".Step_Done := TRUE;
    "DB_RecipeData".PLC_State := 2; // WAIT_CONFIRM
    // DO NOT advance — hold current outputs
    // Agitator keeps running, temperature maintains
END_IF;

// When PC confirms:
IF "DB_RecipeData".Confirm_Step AND "DB_RecipeData".PLC_State = 2 THEN
    // Verify the confirmation matches current step (safety!)
```

```
IF "DB_RecipeData".Confirm_Phase_ID = current_phase
  AND "DB_RecipeData".Confirm_Step_ID = current_sub_step THEN

  // Reset flags
  "DB_RecipeData".Step_Done := FALSE;
  "DB_RecipeData".Confirm_Step := FALSE;

  // ADVANCE to next step
  advance_to_next_step();
  "DB_RecipeData".PLC_State := 1; // Back to EXECUTING
END_IF;
END_IF;
```

Frontend Changes (x62-MixingControlV2.vue)

What the Operator Sees:

Step 5/13 – Phase p0030, Sub-Step 10
Action: [20010] Dissolve Ingredient
Material: Potassium Sorbate

STEP EXECUTION COMPLETE

Temperature: 60.2°C / 60.0°C
Agitator: 1500 RPM
Duration: 5:00 / 5:00

● CONFIRM STEP DONE & ADVANCE

Button Behavior:

1. **Button is DISABLED** while PLC_State = 1 (Executing) — the step is still running
2. **Button turns GREEN** when PLC_State = 2 (Wait_Confirm) — step is done, waiting for operator
3. **Operator clicks** → Frontend publishes MQTT:

```
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10,
  "timestamp": "2026-05-10T17:55:00"
}
```

4. PLC receives → validates → advances → PLC_State goes back to 1
 5. Frontend sees the state change and the next step starts highlighting
-

MQTT Message Flow

Step Execution (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Executing
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": false,
  "Step_Timer": 245        ← Countdown
}
```

Step Ready for Confirm (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 2,           ← Wait Confirm
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": true,        ← Step finished!
  "Step_Timer": 0
}
```

Operator Confirms (Frontend → PLC):

```
Topic: mixing/plant/1/step_confirm
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10
}
```

PLC Advances (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Back to Executing
  "Phase_ID": "p0030",
  "Step_ID": 20,           ← Next step!
  "Step_Done": false,
  "Step_Timer": 300
}
```

Safety Rules

Rule	Description
Phase/Step Echo	Frontend MUST echo back which Phase+Step it's confirming. PLC rejects if mismatch.
Double-click Guard	Frontend disables the confirm button for 2 seconds after click to prevent double-advance.

Heartbeat Still Active	If PC_Heartbeat is lost while in WAIT_CONFIRM, PLC holds position indefinitely (safe state).
Timeout Warning	If WAIT_CONFIRM exceeds 10 minutes, PLC can trigger a warning alarm (but does NOT auto-advance).
Manual Steps	For ActionCode 21010 (Manual Add), the confirm button serves double duty: operator scans the ingredient AND confirms.

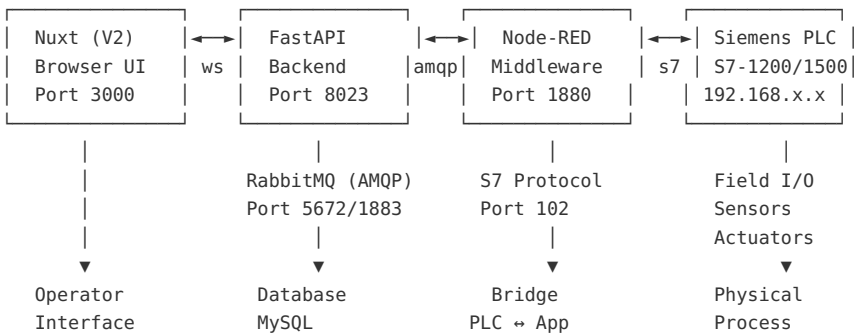
Summary of Changes Required

Layer	What to Change
PLC DB1780	Add Step_Done, Confirm_Step, Confirm_Phase_ID, Confirm_Step_ID fields
PLC FB1780	After step done → set State=2, hold outputs. On Confirm → validate → advance
Node-RED	Add S7-Out node to write Confirm_Step + IDs when MQTT confirm arrives
Frontend	Add “Confirm Step Done” button that enables only when PLC_State = 2
Mock Simulator	Update to wait for confirm MQTT message before advancing

Final Workflow — PLC Mixing Control System (V2)

Version: 1.0 — May 2026
System: xMixing Control — Mitr Phol
Architecture: PLC as Executor, PC as Supervisor with Step Confirmation

1. System Architecture



Role of Each Component

Component	Role	Stateful?
	Executes steps, holds	

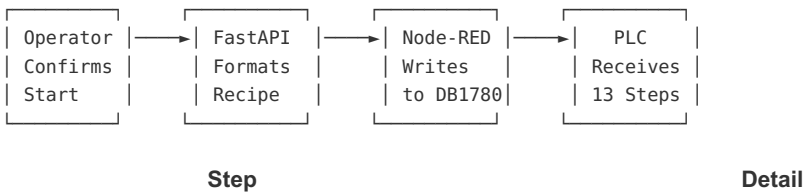
PLC	state, controls outputs (agitator, heater, valves)	Yes — State Master
Node-RED	Bridges PLC ↔ RabbitMQ. Polls PLC every 500ms, publishes changes	× No — Stateless bridge
FastAPI	Formats recipes, logs to DB, broadcasts WebSocket to UI	× No — Event-driven
Nuxt (V2)	Displays live state, provides confirm buttons for operator	× No — Stateless viewer
RabbitMQ	Message broker between all components	× No — Transport only

2. Complete Workflow — Step by Step

Phase A: Batch Setup (Before Production)

Step A1	→	Step A2	→	Step A3	→	Step A4
Planning		PreBatch Weighing		Check Verify		Navigate to Mixing
Step	Who		Action		System	
A1	Planner		Creates Production Plan with SKU, batch size, plant assignment		Nuxt → FastAPI → MySQL	
A2	Warehouse		Weighs prebatch ingredients per plan, scans labels		Nuxt → FastAPI → MySQL	
A3	Operator		Opens “Check for Production” page, verifies all items scanned		Nuxt reads MySQL	
A4	Operator		Clicks “Start Production” → redirected to Mixing Control V2 page		Nuxt navigation	

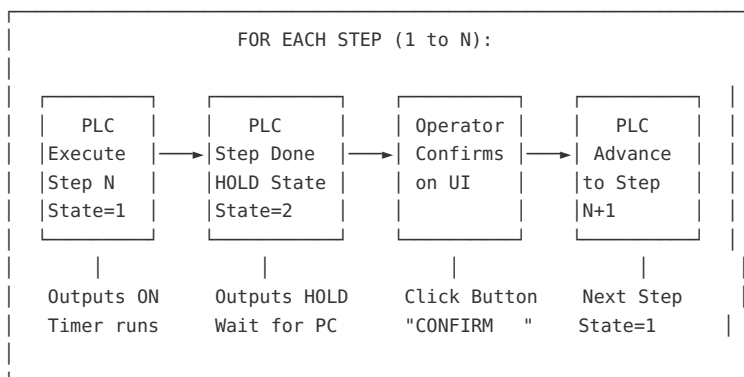
Phase B: Recipe Download to PLC



B1	Operator sees “Confirm Production Start” dialog with Batch ID, SKU, Batch Size
B2	Operator clicks CONFIRM START PRODUCTION
B3	Frontend calls <code>POST /plc/send-recipe/{batch_id}</code>
B4	FastAPI queries <code>sku_steps</code> table → builds DB1780 payload (32 phases × 8 steps)
B5	FastAPI sends payload to Node-RED HTTP endpoint
B6	Node-RED writes recipe array to PLC DB1780 via S7 protocol
B7	Node-RED writes <code>RecipeReady = TRUE</code> to PLC
B8	FastAPI sends <code>StartCmd = TRUE</code> via Node-RED → PLC
B9	PLC validates recipe, sets <code>PLC_State = 1</code> (Executing)

Phase C: Step Execution Cycle (Repeats for Every Step)

This is the **core loop** — it repeats for all 13 steps (or however many the SKU has).



C1. PLC Executes Step (State = 1: EXECUTING)

PLC reads: `DB1780.Processes[current_phase].Steps[current_step]`

PLC sets outputs:

- `Agitator_SP` = `step.AgitatorRPM`
- `HighShear_SP` = `step.HighShearRPM`
- `Temperature_SP` = `step.Temperature`

PLC starts timer:

- `Step_Timer` counting down from `step.StepTime`

PLC publishes telemetry every 500ms via Node-RED → MQTT:

```

{
  "PLC_State": 1,
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": false,
  "Step_Timer": 245,           ← countdown
  "MixTank_Temp_Act": 59.8,
  "Agitator_Act": 1500
}

```

Frontend shows: - Current step highlighted in blue with pulse animation - Live actual values (temp, RPM, weight) updating in real-time - Timer countdown displayed - Confirm button is **DISABLED** (greyed out)

C2. Step Conditions Met → PLC Holds (State = 2: WAIT_CONFIRM)

When timer reaches 0 (or weight/temp condition met):

PLC sets:

- PLC_State = 2 (Wait for Confirm)
- Step_Done = TRUE
- Outputs HOLD at current setpoints (agitator keeps spinning)

PLC publishes:

```
{
  "PLC_State": 2,
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": true,      ← step finished!
  "Step_Timer": 0
}
```

Frontend shows: - Step row turns **GREEN** with "Step Complete" indicator - Big **CONFIRM STEP DONE** button becomes **ENABLED** - Actual values shown next to setpoints for operator verification: -

Temperature: 60.2°C / 60.0°C - Agitator: 1500 RPM / 1500 RPM

C3. Operator Confirms (Frontend → PLC)

Operator visually checks:

- ✓ Temperature reached setpoint?
- ✓ Weight matches requirement?
- ✓ Timer completed?
- ✓ For manual steps: ingredient scanned and added?

Operator clicks: "CONFIRM STEP DONE "

Frontend publishes MQTT:

```
Topic: mixing/plant/1/step_confirm
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",    ← echo back which step
  "Confirm_Step_ID": 10,         ← safety: PLC validates this
  "operator": "cj",
  "timestamp": "2026-05-10T17:55:00"
}
```

Node-RED receives → writes to PLC:

```
DB1780.Confirm_Step = TRUE
DB1780.Confirm_Phase_ID = "p0030"
DB1780.Confirm_Step_ID = 10
```

Frontend also: - Logs the confirmation to FastAPI → MySQL (mixing_batch_step_log) - Records actual values at time of confirmation - Disables confirm button for 2 seconds (double-click guard)

C4. PLC Validates and Advances

PLC checks:

```
IF Confirm_Step = TRUE
```

```
    AND Confirm_Phase_ID matches current phase
    AND Confirm_Step_ID matches current step
THEN:
    → Reset: Step_Done = FALSE, Confirm_Step = FALSE
    → Advance: Current_Step += 1 (or next phase if end of phase)
    → PLC_State = 1 (back to Executing)
    → Start executing next step immediately
ELSE:
    → Reject: Error_Code = 3 (Phase/Step mismatch)
    → Stay in State 2
END_IF
```

Phase D: Special Step Types

D1. Manual Addition Steps (ActionCode 21010)

```
PLC executes step → PLC_State = 2 immediately (no timer)
↓
Operator physically adds ingredient to tank
↓
Operator scans ingredient barcode on UI (verification)
↓
UI validates scan matches re_code for this step
↓
Confirm button enables → Operator clicks CONFIRM
↓
PLC advances to next step
```

D2. QC Recording Steps (Brix/pH Required)

```
PLC executes step → timer completes → PLC_State = 2
↓
UI detects: step.BrixRecord = TRUE or step.PhRecord = TRUE
↓
QC Dialog pops up: "Enter Actual Brix: ____ Enter Actual pH: ____"
↓
Operator enters QC values → clicks CONFIRM
↓
Frontend logs QC values to MySQL + sends confirm to PLC
↓
PLC advances to next step
```

D3. Long Duration Steps (e.g., Pasteurize 300 min)

```
PLC executes step → timer counts from 300:00 to 0:00
↓
During execution: UI shows live countdown + actual temp
↓
Timer reaches 0 → PLC_State = 2 (Wait Confirm)
↓
Operator verifies pasteurization complete → clicks CONFIRM
↓
PLC advances to next step
```

Phase E: Batch Completion

Last step (Step 13/13) confirmed by operator
↓
PLC sets: PLC_State = 4 (DONE), Batch_Complete = TRUE
↓
PLC sets all outputs to safe state:
Agitator_SP = 0, HighShear_SP = 0, Temperature_SP = 0
↓
Frontend shows: " BATCH COMPLETE!" banner
↓
FastAPI updates production_batch status = "Completed" in MySQL
↓
Operator can navigate back to production planning

3. Recovery Workflow (PC/App Crash)

Normal Operation	PC Dies at Step 7	PC Reboots
Step 5: EXECUTING	PLC detects heartbeat	App starts
Step 6: CONFIRM	loss after 5 seconds	
Step 7: EXECUTING	PLC continues Step 7 if AUTO (keeps timer)	App reads PLC: "Batch: P260309" "Step 7, State=2"
	Step 7 timer done	UI jumps to Step 7
	PLC_State = 2 (HOLD)	Shows CONFIRM button
	Waits for operator...	Operator confirms
		Step 8 starts

Key Rule: The PLC **never loses its place**. It holds Current_Phase, Current_Step, and Batch_ID in retentive memory (DB1780). No matter how many times the PC crashes, the PLC keeps the recipe and the progress.

4. PLC State Diagram

```
```text (Mermaid Diagram) stateDiagram-v2 [*] --> IDLE: Power On IDLE --> EXECUTING: StartCmd + RecipeReady EXECUTING --> WAIT_CONFIRM: Step conditions met WAIT_CONFIRM --> EXECUTING: Operator Confirm (advance) EXECUTING --> DONE: Last step confirmed

EXECUTING --> PAUSED: PauseCmd
WAIT_CONFIRM --> PAUSED: PauseCmd
PAUSED --> EXECUTING: StartCmd (resume)

EXECUTING --> IDLE: AbortCmd
WAIT_CONFIRM --> IDLE: AbortCmd
PAUSED --> IDLE: AbortCmd

DONE --> IDLE: New batch loaded

5. MQTT Topic Map
```

Topic	Direction	Purpose
`MIX-01-PUT`	PC → PLC	Heartbeat + current step info (every 1s)
`MIX-01-READ`	PLC → PC	Telemetry readback (every 500ms)
`mixing/plant/1/status`	PLC → PC	State changes, step complete events
`mixing/plant/1/cmd`	PC → PLC	START, PAUSE, ABORT commands
`mixing/plant/1/step_confirm`	PC → PLC	Step confirmation with Phase/Step echo
`mixing/plant/1/recipe`	PC → PLC	Recipe download (one-time at batch start)

---

## ## 6. Database Tables Involved

Table	Purpose
`production_plans`	Batch planning with SKU, size, plant
`production_batches`	Individual batch records
`sku_steps`	Master recipe steps per SKU
`prebatch_items`	Weighed ingredients per batch
`mixing_batch_step_log`	**Step-by-step execution log** with timestamps, actuals, operator

### ### mixing\_batch\_step\_log Schema:

Column	Type	Description
id	INT	Auto-increment
batch_id	VARCHAR(20)	Batch reference
phase_number	VARCHAR(10)	Phase (e.g., p0030)
sub_step	INT	Step within phase
action_code	VARCHAR(10)	Action executed
started_at	DATETIME	When PLC started this step
confirmed_at	DATETIME	When operator confirmed
confirmed_by	VARCHAR(50)	Operator username
temp_actual	FLOAT	Actual temp at confirmation
weight_actual	FLOAT	Actual weight at confirmation
agitator_actual	FLOAT	Actual RPM at confirmation
brix_actual	FLOAT	Brix reading (if QC step)
ph_actual	FLOAT	pH reading (if QC step)
duration_sec	INT	Actual step duration
status	VARCHAR(20)	completed / skipped / error

---

## ## 7. File Map – What Goes Where

```

x2512001-mitrPhol-x31-xMixingControl-site/ | — x9000-Concept/ | — Final_Workflow.md ← THIS
FILE | — PLC_Recovery_Workflow.md ← Recovery concept | — Step_Confirmation_Concept.md ←
Confirm logic detail | — Production_Deployment_Guide.md ← PLC code + Node-RED setup | —
Implementation_Plan.md ← 3-phase dev plan | — s7_1200_db_design.md ← DB1780 structure | —
x3101-app/ | — x3101-0110-frontEnd/app/pages/ | | — x61-MixingControl.vue ← Original (PC-
driven) | | — x62-MixingControlV2.vue ← NEW (PLC-master, stateless) | | — x3101-0210-
backEnd/x0201-fastAPI/ | — main.py ← FastAPI entry point | — routers/router_plc.py ← PLC recipe
endpoints | — plc_datablock.py ← DB1780 serializer | — plc_interface.py ← DB100/DB200 models
| — mock_plc.py ← Quick 5-step simulator | — mock_plc_full.py ← Full production simulator | —
x3109-locMqtt/ ← RabbitMQ Docker config | — x3112-nodeRed/ ← Node-RED flows

```

---

## ## 8. Production Readiness Checklist

```

PLC (TIA Portal)
- [] Create UDT_ProcessStep, UDT_Process
- [] Create DB1780 (Optimized Access = FALSE)
- [] Create FB1780 with execute → hold → confirm → advance logic
- [] Add Step_Done, Confirm_Step, Confirm_Phase_ID, Confirm_Step_ID to DB1780
- [] Map analog I/O tags (temperature, weight, RPM sensors)
- [] Download and test with watch table
- [] Record exact byte offsets for Node-RED

```

```

Node-RED
- [] Install node-red-contrib-s7 + node-red-contrib-amqp
- [] Configure S7 connection to PLC
- [] Create polling flow (read telemetry every 500ms)
- [] Create change-detection function
- [] Create RabbitMQ publish node
- [] Create recipe write flow (HTTP → S7)
- [] Create confirm relay flow (MQTT → S7)
- [] Create heartbeat relay flow

```

```

FastAPI Backend
- [] Update /plc/send-recipe to POST to Node-RED
- [] Add RabbitMQ consumer for plc.mixing.state
- [] Add WebSocket broadcast to Nuxt
- [] Add mixing_batch_step_log table + insert on step confirm
- [] Add heartbeat publisher (toggle every 1s)

```

```

Nuxt Frontend (x62-MixingControlV2.vue)
- [x] Stateless UI – reacts to PLC telemetry only
- [x] Recovery on mount (reads PLC state on page load)
- [x] Recipe download on Confirm Start
- [] Add "CONFIRM STEP DONE" button (enabled when PLC_State = 2)
- [] Add QC dialog for Brix/pH steps
- [] Add step confirmation MQTT publish with Phase/Step echo
- [] Add double-click guard (2s disable after confirm)
- [] Log confirmation to backend API

```

```

Testing
- [] Run mock_plc_full.py – verify all 13 steps appear in UI
- [] Test step confirmation flow end-to-end
- [] Pull-the-plug test: kill FastAPI at Step 7, restart, verify recovery
- [] Manual step test: verify PLC holds on ActionCode 21010
- [] QC step test: verify Brix/pH dialog appears and records values
- [] Double-confirm test: verify rapid clicks don't skip steps

```

---

# Work Instruction & Data Flow (xMixing Control)

เอกสารระบุวิธีการทำงานและคู่มือปฏิบัติงาน (Work Instruction - WI) สำหรับ Operator ตลอดจนการทำงานร่วมกันระหว่าง  
**\*\*Scanner\*\* ↔ \*\*xMixing App\*\* ↔ \*\*PLC\*\***

---

## 1. Data Flow Architecture (การรับส่งข้อมูล)

ระบบจะถูกขับเคลื่อนด้วย **\*\*"คน" เป็นผู้คุมจังหวะ\*\*** และ **\*\*"PLC" เป็นผู้ลงมือทำ\*\***

```text (Mermaid Diagram)

sequenceDiagram

participant SC as  Barcode Scanner

participant APP as  xMixing App

participant DB as Database

participant PLC as  PLC (S7-1200)

Note over SC,APP: 1. Preparation & Validate

SC->>APP: Scan Batch QR/Barcode

APP->>DB: ตรวจสอบแผนผลิต & ดึงสูตร (Recipe)

DB-->>APP: ส่งคืนสูตรที่ละ Step ไปโชว์หน้าจอ HMI

Note over APP,PLC: 2. Execution (Step-by-Step Handshake)

APP->>PLC:  ยิง Send Step 1 (อุณหภูมิ, น้ำหนัก, เวลา)

APP->>PLC:  ส่งคำสั่ง COMMAND: START

Loop During execution

PLC-->>APP: Telemetry (RPM, Temp, Weight ปัจจุบัน) ทุกๆ 1s

end

PLC->>APP: แจ้งจบการทำงาน STATUS: STEP 1 COMPLETE

APP->>DB: Auto-save ประวัติการทำงาน Step 1 (รับน้ำหนัก, ค่าจริงเก่าใหม่)

Note over APP,PLC: ระบบจะวิ่งไป Step 2 จนจบ Phase (หรือรอ Operator กดเริ่ม Phase ใหม่)

Note over APP,SC: 3. Manual QC & Validation (บาง Step)

APP->>SC:  บังคับ Scan ยืนยันวัตถุดิบ (ถ้าจำเป็น)

APP->>APP:  บังคับกรอก Brix / pH (ในขั้นตอน x1030/x1040)

APP->>DB: Save Production & QC Record

2. Work Instruction (WI) ขั้นตอนการปฏิบัติงานของ Operator

ขั้นตอนที่ 1: การรับแผนและตั้งค่าเริ่มต้น (Preparation)

- Operator เข้าสู่ระบบ (Login) ที่หน้าจอ HMI บนเครื่องจักร **Mixing Plant**
- ไปที่เมนู “**Check for Production (x60)**”
- เตรียมใบสั่งผลิต (Job Order)
- ยก **Scanner** ขึ้นสแกน QR Code/Barcode ที่อยู่บนใบสั่งผลิต
 - ถ้าระบบหา Batch เจอ -> จะแสดงรายละเอียดสูตร (SKU), น้ำหนักที่ต้องการผลิตทางฝั่งขวา
 - ช่องสีเทา/สีฟ้าจะแยก Phase ให้อ่านง่าย
- ตรวจสอบข้อมูลสูตรและปริมาณให้ตรงกับใบจริง
- กดปุ่ม **[GO TO PRODUCTION CONTROL]** ด้านขวาล่างเพื่อเข้าสู่หน้าคุมเครื่อง (x61)

ขั้นตอนที่ 2: เริ่มต้นรันเครื่องจักร (Start Batch)

- ในหน้า **Mixing Control (x61)** หน้าจอจะแสดงหน้าปัดควบคุม (Gauges) เป็นค่า 0
- ตรวจสอบสถานะว่าป้ายข้างๆ โฉว์คำว่า  **PLC CONNECTED**
- แจ้งพนักงานห้องเตรียมวัตถุดิบ หรือตรวจเช็คส่วนหน้าให้พร้อม (ระบบพร้อมไหลدنน้ำ / น้ำตาลทราย)
- กดปุ่ม  **[START]** สีเขียวตรง Command Center
- แอปฯ จะโยน **Step ที่ 1** ลงไปหา PLC และ PLC จะเริ่มทำงานทันที (ดึงน้ำ, ดันอุณหภูมิ)
- Operator ฝ้าดูหน้าจอบอกสถานะ **Actual vs Setpoint**
 - ไม่ต้องกด Start ใดๆ Step ภายใน Phase — PLC กับ App จะคุยกันเองว่าส่งของถูกไหม

II ขั้นตอนที่ 3: กรณีฉุกเฉินหรือหยุดกลางคัน (Pause / Abort)

- หากต้องการดูอาการ หรือตรวจเช็คคุณภาพถัง และ II [PAUSE]
 - PLC จะพักการปั่น และหยุดนับเวลา แต่จะไม่เคลียร์ค่า
 - หากพร้อมทำต่อ กด II อีกครั้ง เพื่อทำต่อจากจุดเดิม
- หากสูตรผิด หรือเครื่องพัง ให้กด [ABORT] สีแดง
 - ระบบจะหยุดจ่ายไฟ และทำการล้างค่าในระบบ Operator ต้องไปยกเลิก Batch และออกหมายเหตุ

ขั้นตอนที่ 4: การเติมส่วนผสมย่อย (Manual Dosing) & ไล่ค่า QC

1. ใน Phase ทาละลายสาร (D1010/D1030) ที่หน้าจอจะโชว์ว่าให้ใส่ส่วนผสมอะไร (เช่น Potassium Sorbate 0.5 kg)
2. Operator หยิบ Scanner มายิงบาร์โค้ดที่ถุงส่วนผสม เพื่อยืนยันว่าหยิบไม่ผิดตัว (Option เสริมในอนาคต)
3. เมื่อ PLC ปล่อยอุณหภูมิถึงจุด Holding หรือ Cooldown (x1030, x1040)
4. เครื่องจะหยุดรอการยืนยัน
5. Operator นำสายวัด หรือดึง Sampling ออกมาวัด Brix และ pH
6. กรอค่าที่วัดได้จริง ลงในช่องสี่เหลี่ยมบน HMI
7. กด [CONFIRM / NEXT_STEP] วิ่งสเตปต่อไป

ขั้นตอนที่ 5: สิ้นสุดกระบวนการ (End Batch)

1. เมื่อครบ 35 Steps หรือจนกว่าครบทุก Phase ระบบจะขึ้นแจ้งพุดอปับ “ BATCH COMPLETE”
2. ระบบจะบันทึก Log ลง Database พร้อมทำรายงาน
3. Operator กดเปิดหน้าปิดล้างถังกวน หรือปั๊มออกไปยังถัง Holding ถัดไป
4. สิ้นสุดงาน (สามารถกดปริ้นท์ Log ท้ายกะได้จากปุ่ม 🖨️ ปริ้นท์ด้านบนขวา)

3. สรุปความรับผิดชอบ (Who does what?)

| ผู้รับผิดชอบ | หน้าที่ |
|----------------|--|
| Operator | สแกน Job Order, กด START, เฝ้าระวังจอ (Monitor), ใส่ส่วนผสมมือ (Manual Dosing), กรอค่า QC, ตัดสินใจกด STOP ฉุกเฉิน |
| 🖥️ xMixing App | โหลดสูตร, แยกเป็น Step, ยิง Step ไป PLC, รับแจ้งจบ Step แล้วอัปโหลดใส่ Database (รับอัตรา 80%), ตรวจสอบ Tolerance |
| 🔌 PLC | อ่านค่าน้ำหนักเครื่องชั่ง, หมุนปั๊ม, จ่ายสตรัมให้ร้อน, คุม High Shear ตาม Step ที่รับมา |
| 📡 Scanner | ลด Human Error ด้วยการสแกนเลขโค้ด / สแกนหลอดส่วนผสมก่อนโยนลงถัง |

🔧 x61 - Mixing Control Page Concept (Plant 1)

หน้านี้คือ หัวใจหลักของการดำเนินงาน (Operation Control) เป็นหน้าจอที่ Operator ใช้คุมเครื่องจักรและตรวจสอบสถานะทุกอย่างระหว่างการผลิต โดยในเฟสนี้เราจะโฟกัสที่ Plant 1 (Mixing Tank 1) เป็นหลัก

1. หัวใจหลักของหน้า x61 (Core Concepts)

- Safety & Clarity FIRST** : ต้องเห็นคำสำคัญชัดเจน (อุณหภูมิ, น้ำหนัก) และมีปุ่มฉุกเฉิน (ABORT) ที่เรียกใช้งานได้ง่าย
- Step-by-Step Handshake** : หน้าเว็บจะเป็นคนจ่ายงาน (Recipe) ที่ละบรรทัดให้ PLC และรอจนกว่า PLC จะทำเสร็จ เติมน้ำเสร็จ แล้วจึงไปต่อ
- Actual vs Setpoint Comparison** : โชว์เปรียบเทียบค่าที่ควรจะเป็น (SP) กับค่าที่วัดได้จริงจากเซนเซอร์ตลอดเวลา
- Mandatory QC Stop** : ถ้าเป็น Step ที่เกี่ยวกับการเย็นลง (Cooldown/Final) ระบบต้องหยุดให้ Operator ตรวจสอบค่า Brix / pH ไม่ขึ้นไม่ให้ออก Batch

2. โครงสร้างหน้าจอ (UI Layout Design)

หน้าจอออกแบบมาให้ Operator เข้าใจง่าย แบ่งเป็นโซนชัดเจน (3 โซนหลัก):

■ Zone A: Header & Status (ด้านบนสุด)

- วัตถุประสงค์: ข้อมูลแผนการผลิต
- ข้อมูลที่แสดง:
 - ชื่อ SKU และรหัสสินค้า (เช่น S77S743200 - Cafe Amazon)
 - หมายเลข Batch และ Target Weight (เช่น 1200kg)
 - เชื่อมต่อ PLC สำเร็จหรือไม่ (แสดงป้าย ● PLC CONNECTED)

■ Zone B: Live Telemetry Dashboard (ตรงกลางจอ)

- วัตถุประสงค์: แสดงสถานะเครื่องจักร Real-time ทุก 1 วินาที
- ข้อมูลที่แสดง (แบ่งเป็น 4 คอลัมน์):
 - Hopper Scale**: น้ำหนักของสารระเหย หรือสารปรุงแต่งที่รอโหลด
 - Mixing Tank (ถังหลัก)**: อุณหภูมิน้ำ, น้ำหนักรวม, ความเร็วใบกวน Agitator
 - High Shear**: ความเร็วใบมีดบดละเอียด และอุณหภูมิมอเตอร์
 - Circulation**: ความเร็ว Flow Pump
- จุดเด่น**: ในส่วนนี้จะมีการแจ้งเตือนง่ายๆ โชว์คำว่า SP: 40°C เป๊ะไว้ข้างๆ Actual: 42°C ถ้าอยู่ในเกณฑ์ $\pm 5^{\circ}\text{C}$ จะเป็นสีเขียว ถ้าเลยเกณฑ์จะเป็นสีแดง

■ Zone C: Operation & Command Center (ด้านล่าง)

- วัตถุประสงค์: การควบคุม และบอกพิกัดว่าถึงขั้นไหนแล้ว
- ข้อมูลที่แสดง:
 - Command Buttons**:
 - ▶ START: จ่ายงาน Step ปัจจุบันให้ PLC รับ
 - ⏸ PAUSE: สั่งให้ PLC ระงับการขยับ ทุกอย่างหยุดหมุน
 - ABORT: เบรกด่วนฉุกเฉิน เคลียร์ระบบ
 - ⏭ Force Next Step: ข้ามสแตปแบบบังคับ (ใช้กรณีฉุกเฉิน/Manual Override)
 - SKU Step List (Accordion)**: แถบกลิ้งที่ไล่ Phase (เช่น Phase A1010) แล้วแสดงว่า PLC ตอนนี้อยู่ที่ขั้นตอนไหน (Highlight) ให้รู้ว่า “กำลังกวนตัวนี้อยู่นะ”

3. 🌀 ลำดับการควบคุมจากหน้า x61 (Operation Flow)

1. **Initiate (โหลดข้อมูล):**
 - เมื่อเข้ามาจากหน้า x60 ระบบจะดึง “ทุก Step” ใต้ตารางเก็บไว้
 - เชื่อมต่อ MQTT รับค่าจาก Plant 1
2. **Execute (การรันแบบปกติ):**
 - Operator กด ▶ START
 - หน้า x61 ส่งข้อมูล Step ที่ 1 เท่านั้น ไปที่: mixing/plant/1/step_cmd
 - รับข้อมูล status: STEP_COMPLETE จาก PLC (วนลูปแบบนี้จนจบ)
3. **The QC Trap (ขั้นตอนบังคับ):**
 - เมื่อ Step ใดไปจนถึงกลุ่ม Cooldown (x1030, x1040)
 - หน้าจอจะค้าง ไม่ Auto-advance
 - มีกล่อง Input ให้กรอก: Actual Brix [] และ Actual pH []
 - ต้องกรอกแล้วกด Save -> ระบบบันทึกลง DB -> ค่อยวิ่งต่อไป
4. **Conclusion (จบงาน):**
 - เมื่อถึงบรรทัดสุดท้าย โชว์ข้อความสำเร็จ พิมพ์เอกสารสรุป Batch Report อัตโนมัติ

4. 🏠 ลอจิกการคุยกับระบบหลังบ้าน (Backend API / Database)

เนื่องจากต้องบันทึกประวัติ (History) หน้า x61 นี้จะเป็นจุดเดียวที่ยิง API ลงฐานข้อมูล:

- **เมื่อทำ Finished 1 Step:**
 - POST /api/records/step
 - Payload: { batch_id, step_id, actual_temp_end, duration_seconds }
- **เมื่อกด Confirm QC Check:**
 - POST /api/records/qc
 - Payload: { batch_id, phase_id, actual_brix, actual_ph }
- **เมื่อรันถึง Step สุดท้าย:**
 - POST /api/batch/complete

สรุป (Conclusion)

หน้า **x61-MixingControl** ถูกออกแบบให้เป็น “HMIอัจฉริยะ” ไม่ใช่แค่หน้าจออมิเตอร์ แต่ทำหน้าที่ผู้ป้อนคำสั่งให้ PLC (S7-1200) ทำงานทีละคำสั่ง (Micro-management) ข้อดีคือลดภาระ Memory ของ PLC และช่วยให้ฝั่ง Software เก็บ Snapshot ข้อมูลทุก Step ได้ละเอียดที่สุด

Barcode Scanning & Verification Workflow

This diagram illustrates the step-by-step logic the application follows when an operator scans a QR code using the handheld scanner. It highlights the error-handling logic for various scanning mistakes, including the Wrong Box handling.

```text (Mermaid Diagram) flowchart TD A([Scanner reads QR Code]) --> B{Is a Batch already loaded?} B --

```

No -> C[Load the scanned Batch] B – Yes -> D{Is scanned code a completely new Batch ID?}

D -- Yes --> E[Switch and Load the new Batch]
D -- No --> F[Verify as an Ingredient Bag]

F --> G[Check local data for matching ingredient]
G -- Match Found --> H([Mark Ingredient as Verified])
G -- No Local Match --> I[Send scan to Backend API]

I --> J{Backend Response}
J -- Success --> K([Mark Ingredient as Verified])

J -- Failure --> L{What did the operator actually scan?}

L -- "The same Batch ID again" --> M([⚠ Yellow Warning:
'You scanned the Batch Label!'])
L -- "Unrecognized Bag for this Batch" --> N([⚠ Red Toast Error:
'Ingredient not found'])
L -- "Bag from a completely different box" --> O([⚠ Big Red Modal:
WRONG BOX!])

O --> P{Operator Decision}
P -- "I want to keep working on my current batch" --> Q([Close Modal & Continue])
P -- "I want to scan a different batch instead" --> R([Start New Batch / Reset])

```

---

#### # PLC & PC Concept and Recovery Workflow

\*Architecture for seamless recovery from App or PC shutdown during mixing execution.\*

To handle scenarios where the App or Computer shuts down abruptly while maintaining the ability to seamlessly recover, the best approach is the **\*\*\*PLC as Executor, PC as Supervisor\*\*\*** concept.

Once the production starts, the PLC must not rely on the PC to feed it instructions step-by-step. Instead, the PLC holds the recipe and manages the state machine, while the PC just watches and logs.

#### ## 1. The Concept (PLC as State Master)

\* **Pre-load the Recipe:** The entire recipe (or up to 50 steps) is downloaded into the PLC's `DB\_Recipe` *before* the batch officially starts.

\* **PLC Manages the Sequence:** The PLC uses an internal variable (e.g., `Current\_Step\_Idx`) to track where it is. It executes Step 1, finishes it, and automatically moves to Step 2 without asking the PC.

\* **PC is just a Viewer/Logger:** The App continuously polls the PLC to read `Current\_Step\_Idx` and logs the actual values (actual temp, time taken, etc.) to the database.

\* **Resume Capability:** Because the PLC knows the `Batch\_ID` and the `Current\_Step\_Idx`, if the PC reboots, the PC simply asks the PLC: *"What Batch are you running, and what step are you on?"* and immediately syncs its UI back to reality.

---

#### ## 2. Required Updates to your Data Block Header

To make recovery work, you need to add a few synchronization variables to your `DB\_Recipe` header:

```

```pascal
// Header Information
Batch_ID      : STRING[20]; // Critical: Links PLC memory to your Database Batch
SKU_ID       : STRING[20];
Total_Steps  : INT;
Current_Step_Idx : INT;      // PLC updates this as it moves forward

```

```
// Handshake & Status
Command_Start      : BOOL;          // PC tells PLC to Start
Command_Pause      : BOOL;          // PC tells PLC to Pause
PC_Heartbeat        : BOOL;          // Toggles every 1s. If PLC doesn't see toggle, PC is
dead.
PLC_State           : INT;           // 0=Idle, 1=Running, 2=Waiting for Manual, 3=Paused,
4=Done
```

3. Normal Workflow (No Crashes)

1. **Download:** Operator selects a Batch. The App writes the Batch_ID and the array of up to 50 steps to the PLC.
 2. **Start:** App sends Command_Start = TRUE.
 3. **Execution:** PLC reads Steps[Current_Step_Idx], sets Agitator, High Shear, and Temp. It waits for the Step_Time to finish.
 4. **Transition:** When the time is up, PLC increments Current_Step_Idx += 1 and executes the next step automatically.
 5. **Logging:** The App notices Current_Step_Idx changed, so it records the completion of the previous step into your mixing_batch_step_log table.
-

4. Recovery Workflow (App / Computer Crashes & Restarts)

Scenario: The PC dies during Step 15.

1. **PLC Reaction (PC_Heartbeat fails):**
 - The PLC detects the PC_Heartbeat stopped toggling.
 - **If the step is automatic** (e.g., mixing for 5 minutes), the PLC *continues* running the step to avoid ruining the product.
 - **If the step requires manual scanning** (like Action 21010 "Manual Add"), the PLC safely goes into a *Waiting* state (keeping agitator on, but timer paused) because the operator needs the PC scanner to continue.
 2. **PC Reboots & App Starts:**
 - The App connects to the PLC and reads the Batch_ID and Current_Step_Idx.
 - **App Logic:** "Ah, the PLC says it is running Batch B20260510-01 at Step 15."
 3. **Syncing the Database:**
 - The App checks its database for B20260510-01.
 - It sees the batch status is still In Progress.
 - It sees logs up to Step 14. It knows Step 15 is currently active.
 4. **Resume UI:**
 - The App instantly routes the operator to the Mixing Control screen for that Batch, highlights Step 15 as "Running", and resumes polling data. **Recovery Complete without any operator input.**
-

5. PLC Communication Protocol: The Best Choice

Since your architecture already utilizes **Node-RED** and **RabbitMQ**, the absolute most robust way to communicate with the PLC is to decouple the PC from the PLC physically using Node-RED as middleware.

Option A: S7 Protocol via Node-RED (Recommended for Siemens S7-1200/1500) * Use the `node-red-contrib-s7` node. * **Why:** It natively reads/writes to the Data Block (DB) based on absolute offsets (e.g., `DB1,X0.0`) without requiring any special code or licenses on the PLC side. It is extremely fast and lightweight.

Option B: OPC UA (Recommended if PLC supports it natively) * Use `node-red-contrib-opcua`. * **Why:** Tag-based and highly standardized. If the PLC tags move, OPC UA doesn't break.

The Ideal Communication Flow: 1. **Polling:** Node-RED connects to the PLC and polls the `DB_Recipe` header every 500ms. 2. **Publishing:** If `Current_Step_Idx` or `PLC_State` changes, Node-RED publishes a JSON message to a **RabbitMQ** topic (e.g., `plc.mixing.state`). 3. **Consuming:** FastAPI listens to RabbitMQ, logs the step completion securely into MySQL (`mixing_batch_step_log`), and broadcasts the live state to Nuxt.

6. Required Changes to Existing Applications

Backend (FastAPI) Updates

1. **Remove State-Machine Logic:** Remove any Python logic that says “*Wait 5 minutes then trigger the next step*”. FastAPI must become stateless; it only reacts to events emitted by the PLC via RabbitMQ.
2. **Add Download Endpoint:** Create a `POST /api/batch/{id}/download-to-plc` endpoint. This query formats the 50-step array and sends it to Node-RED to write to the PLC `DB_Recipe`.
3. **RabbitMQ Consumer Worker:** Create a background worker in FastAPI to consume PLC state changes from RabbitMQ and update the MySQL database instantly.

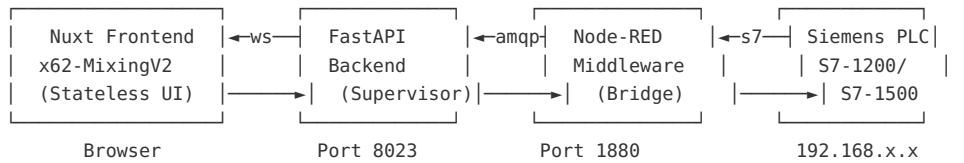
Frontend (Nuxt) Updates

1. **Make UI Stateless:** The UI must stop driving the process. Remove the manual “Next Step” buttons for any automated mixing/heating steps.
 2. **WebSocket / SSE Integration:** The frontend should connect to FastAPI via WebSocket. When the PLC moves to Step 15, the frontend instantly receives the event and visually highlights Step 15 in the UI.
 3. **Recovery on Mount (onMounted):** When the operator navigates to the Mixing Control page, the code should immediately fetch the current `PLC_State` and `Current_Step_Idx`. If the batch is already running, the UI simply jumps to that step and resumes monitoring.
 4. **Heartbeat Signal:** The Nuxt frontend (or FastAPI) must implement a loop that toggles the `PC_Heartbeat` bit in the PLC every 1-2 seconds so the PLC knows the PC is alive.
-

Production Deployment Guide — PLC Mixing Control V2

This document explains **exactly** what needs to happen on each layer of the system to move from our tested mock environment to a real, running production line.

Architecture Overview



Data Flow: 1. **Download:** Nuxt → FastAPI → Node-RED → PLC (write recipe once) 2. **Telemetry:** PLC → Node-RED (poll 500ms) → RabbitMQ → FastAPI → Nuxt (live state) 3. **Heartbeat:** FastAPI → RabbitMQ → Node-RED → PLC (toggle bit every 1s)

PART A: PLC Code (TIA Portal)

Important: All code below is in SCL (Structured Control Language) for Siemens TIA Portal V16+. Your PLC engineer should create these in the TIA Portal project.

A1. User-Defined Types (UDTs)

Create these two UDTs first. They define the shape of every recipe step and the overall recipe.

```
// =====  
// UDT: "UDT_ProcessStep"  
// Size: ~136 bytes per step  
// =====  
TYPE "UDT_ProcessStep"  
VERSION : 0.1  
STRUCT  
    StepNo      : Int;           // Sub-step number (1, 2, 3...)  
    ActionCode  : Int;           // Action code (10010=Setup, 21010=Manual Add, etc.)  
    ReCode      : String[25];    // Recipe ingredient code  
    SapCode     : String[20];    // SAP Material code  
    Destination : Int;           // Target vessel (0=MixTank, 1=Hopper...)  
    Require     : Real;          // Required weight (kg)  
    LowTol      : Real;          // Low tolerance (kg)  
    HighTol     : Real;          // High tolerance (kg)  
    Temperature : Real;          // Setpoint temperature (°C)  
    TempLow     : Real;          // Temp low limit  
    TempHigh    : Real;          // Temp high limit  
    AgitatorRPM : Real;          // Agitator speed setpoint (RPM)  
    HighShearRPM : Real;         // High-shear mixer speed (RPM)  
    StepTime    : Int;           // Step duration (seconds)  
    StepTimerCtl : Int;          // Timer control mode (0=none, 1=auto, 2>manual)  
    SetupStep   : Int;           // Setup step flag  
    Condition   : Int;           // Step condition code  
    QcTemp      : Bool;          // Requires QC temperature check  
    RecordSteam : Bool;          // Record steam pressure  
    RecordCTW   : Bool;          // Record CTW data  
    BrixRecord  : Bool;          // Requires Brix recording  
    PhRecord    : Bool;          // Requires pH recording  
    MasterStep   : Bool;          // Is a master step (phase start)  
    StepActive  : Bool;          // Step has valid data (not empty)  
    BrixSP      : Real;          // Brix setpoint  
    PhSP        : Real;          // pH setpoint  
END_STRUCT;
```

```

END_TYPE

// =====
// UDT: "UDT_Process" (Phase)
// Each process contains up to 8 steps
// =====
TYPE "UDT_Process"
VERSION : 0.1
STRUCT
    ProcessNo      : Int;           // Phase number (10, 20, 30...)
    PhaseID        : Int;           // Phase ID from DB
    StepCount      : Int;           // Number of active steps in this phase
    ProcessActive   : Bool;         // Phase has valid data
    Steps          : Array[0..7] of "UDT_ProcessStep";
END_STRUCT;
END_TYPE

```

A2. Data Block — DB1780 “DB_RecipeData”

This is the main recipe data block that the PC writes to and the PLC reads from.

```

// =====
// DB1780: "DB_RecipeData"
// The PC writes recipe data here BEFORE starting
// =====
DATA_BLOCK "DB_RecipeData"
{ S7_Optimized_Access := 'FALSE' }
VERSION : 0.1
STRUCT
    // — Header (written by PC) —
    PlanID      : String[30];
    BatchID     : String[20];
    SkuID       : String[20];
    SkuName     : String[50];
    PlantID     : Int;
    BatchSize   : Real;
    ProcessCount : Int;           // Number of active phases

    // — Control Flags (written by PC) —
    RecipeReady : Bool;           // PC sets TRUE after download complete
    StartCmd    : Bool;           // PC sets TRUE to start batch
    PauseCmd    : Bool;           // PC sets TRUE to pause
    AbortCmd    : Bool;           // PC sets TRUE to emergency stop
    PC_Heartbeat : Bool;          // PC toggles every 1 second

    // — Status (written by PLC) —
    PLC_State    : Int;           // 0=Idle, 1=Running, 2=WaitManual, 3=Paused, 4=Done,
9=Error

    Current_Process : Int;       // Current active phase index (0-31)
    Current_Step     : Int;       // Current active step index within phase (0-7)
    Current_Step_Flat : Int;      // Flat step counter across all phases (1-based)
    Step_Timer_Act   : Int;       // Remaining seconds on current step timer
    Step_Complete    : Bool;      // Pulse: PLC sets TRUE when a step finishes
    Batch_Complete   : Bool;      // TRUE when all steps are done
    Error_Code       : Int;       // 0=None, 1=Heartbeat Lost, 2=Timeout

    // — Actuals (written by PLC from I/O) —
    MixTank_Temp_Act : Real;      // Actual tank temperature
    MixTank_Weight_Act : Real;    // Actual tank weight
    Agitator_Act     : Real;      // Actual agitator RPM
    HighShear_Act    : Real;      // Actual high-shear RPM

```



```

    Hopper_Weight_Act : Real;      // Actual hopper weight
    Brix_Act           : Real;      // Actual Brix reading
    PH_Act             : Real;      // Actual pH reading

    // — Recipe Array (32 phases × 8 steps) —
    Processes          : Array[0..31] of "UDT_Process";
END_STRUCT;
BEGIN
END_DATA_BLOCK

```

△ **CRITICAL:** Set `S7_Optimized_Access := 'FALSE'` so Node-RED can read/write using absolute byte offsets (e.g., `DB1780, DBD100`).

A3. Function Block — FB1780 “FB_MixingSequencer”

This is the **brain** of the system. It runs in a cyclic OB (e.g., OB1 or OB35) and manages the step-by-step execution.

```

// =====
// FB1780: "FB_MixingSequencer"
// Main sequencer logic – runs in OB1 every scan cycle
// =====
FUNCTION_BLOCK "FB_MixingSequencer"
VERSION : 0.1

VAR_INPUT
    // Analog inputs from field instruments
    Act_MixTank_Temp   : Real;
    Act_MixTank_Weight : Real;
    Act_Agitator_RPM   : Real;
    Act_HighShear_RPM  : Real;
    Act_Hopper_Weight  : Real;
    Act_Brix           : Real;
    Act_PH             : Real;
END_VAR

VAR_OUTPUT
    // Analog outputs to field devices
    SP_Agitator_RPM    : Real;
    SP_HighShear_RPM   : Real;
    SP_Temperature     : Real;
END_VAR

VAR
    stepTimer          : TON;      // IEC Timer for step duration
    heartbeatTimer     : TON;      // Heartbeat watchdog timer
    lastHeartbeat      : Bool;     // Previous heartbeat state
    flatStepCounter    : Int;      // Running count across all phases
END_VAR

VAR_TEMP
    curProcess         : Int;
    curStep            : Int;
    stepData           : "UDT_ProcessStep";
    stepTimeSec        : Time;
END_VAR

BEGIN

    // == 1. UPDATE ACTUALS ==

```

```

"DB_RecipeData".MixTank_Temp_Act := Act_MixTank_Temp;
"DB_RecipeData".MixTank_Weight_Act := Act_MixTank_Weight;
"DB_RecipeData".Agitator_Act := Act_Agitator_RPM;
"DB_RecipeData".HighShear_Act := Act_HighShear_RPM;
"DB_RecipeData".Hopper_Weight_Act := Act_Hopper_Weight;
"DB_RecipeData".Brix_Act := Act_Brix;
"DB_RecipeData".PH_Act := Act_PH;

// == 2. HEARTBEAT WATCHDOG ==
// If PC_Heartbeat stops toggling for 5 seconds → PC is dead
IF "DB_RecipeData".PC_Heartbeat <> #lastHeartbeat THEN
    #lastHeartbeat := "DB_RecipeData".PC_Heartbeat;
    #heartbeatTimer(IN := FALSE, PT := T#5s); // Reset timer
END_IF;
#heartbeatTimer(IN := TRUE, PT := T#5s);

IF #heartbeatTimer.Q AND "DB_RecipeData".PLC_State = 1 THEN
    // PC lost! But don't stop the batch – just log the error
    "DB_RecipeData".Error_Code := 1; // Heartbeat lost
    // If current step is manual (ActionCode 21010), pause
    #curProcess := "DB_RecipeData".Current_Process;
    #curStep := "DB_RecipeData".Current_Step;
    #stepData := "DB_RecipeData".Processes[#curProcess].Steps[#curStep];
    IF #stepData.ActionCode = 21010 THEN
        "DB_RecipeData".PLC_State := 2; // WaitManual – safe pause
    END_IF;
    // Automatic steps (mixing, heating) CONTINUE running
END_IF;

// == 3. COMMAND HANDLING ==
// Start
IF "DB_RecipeData".StartCmd AND "DB_RecipeData".RecipeReady THEN
    IF "DB_RecipeData".PLC_State = 0 OR "DB_RecipeData".PLC_State = 3 THEN
        "DB_RecipeData".PLC_State := 1; // Running
        "DB_RecipeData".StartCmd := FALSE;
        "DB_RecipeData".Error_Code := 0;
        IF "DB_RecipeData".Current_Process = 0 AND "DB_RecipeData".Current_Step = 0 THEN
            #flatStepCounter := 1;
        END_IF;
    END_IF;
END_IF;

// Pause
IF "DB_RecipeData".PauseCmd THEN
    "DB_RecipeData".PLC_State := 3; // Paused
    "DB_RecipeData".PauseCmd := FALSE;
END_IF;

// Abort
IF "DB_RecipeData".AbortCmd THEN
    "DB_RecipeData".PLC_State := 0; // Idle
    "DB_RecipeData".AbortCmd := FALSE;
    "DB_RecipeData".Current_Process := 0;
    "DB_RecipeData".Current_Step := 0;
    #flatStepCounter := 0;
    SP_Agitator_RPM := 0.0;
    SP_HighShear_RPM := 0.0;
    SP_Temperature := 0.0;
END_IF;

// == 4. MAIN SEQUENCER (only when Running) ==
IF "DB_RecipeData".PLC_State = 1 THEN

```

```

#curProcess := "DB_RecipeData".Current_Process;
#curStep    := "DB_RecipeData".Current_Step;

// Safety: check bounds
IF #curProcess > 31 OR NOT "DB_RecipeData".Processes[#curProcess].ProcessActive THEN
    "DB_RecipeData".PLC_State := 4; // Done
    "DB_RecipeData".Batch_Complete := TRUE;
    SP_Agitator_RPM := 0.0;
    SP_HighShear_RPM := 0.0;
    RETURN;
END_IF;

// Get current step data
#stepData := "DB_RecipeData".Processes[#curProcess].Steps[#curStep];

// — Apply setpoints to outputs —
SP_Agitator_RPM := #stepData.AgitatorRPM;
SP_HighShear_RPM := #stepData.HighShearRPM;
SP_Temperature := #stepData.Temperature;

// — Update flat counter for UI —
"DB_RecipeData".Current_Step_Flat := #flatStepCounter;

// — MANUAL STEP (ActionCode 21010): Wait for operator —
IF #stepData.ActionCode = 21010 THEN
    "DB_RecipeData".PLC_State := 2; // WaitManual
    // The PC will set StartCmd=TRUE again when operator scans/confirms
    RETURN;
END_IF;

// — TIMED STEP: Run step timer —
IF #stepData.StepTime > 0 THEN
    #stepTimeSec := INT_TO_TIME(#stepData.StepTime * 1000);
    #stepTimer(IN := TRUE, PT := #stepTimeSec);
    "DB_RecipeData".Step_Timer_Act :=
        TIME_TO_INT(#stepTimeSec - #stepTimer.ET) / 1000;

    IF NOT #stepTimer.Q THEN
        RETURN; // Still counting, come back next scan
    END_IF;
    // Timer finished – reset and advance
    #stepTimer(IN := FALSE, PT := #stepTimeSec);
END_IF;

// — STEP COMPLETE → advance —
"DB_RecipeData".Step_Complete := TRUE; // Pulse for Node-RED to catch

// Move to next step in this phase
IF #curStep + 1 < "DB_RecipeData".Processes[#curProcess].StepCount THEN
    "DB_RecipeData".Current_Step := #curStep + 1;
    #flatStepCounter := #flatStepCounter + 1;
ELSE
    // Move to next phase
    "DB_RecipeData".Current_Step := 0;
    "DB_RecipeData".Current_Process := #curProcess + 1;
    #flatStepCounter := #flatStepCounter + 1;

    // Check if we've exceeded all active phases
    IF #curProcess + 1 >= "DB_RecipeData".ProcessCount THEN
        "DB_RecipeData".PLC_State := 4; // Done
        "DB_RecipeData".Batch_Complete := TRUE;
    END_IF;
END_IF;

```

```

        SP_Agitator_RPM := 0.0;
        SP_HighShear_RPM := 0.0;
    END_IF;
END_IF;

END_FUNCTION_BLOCK

```

A4. Calling FB1780 in OB1

In your main program (OB1), create an instance DB and call the function block:

```

// In OB1 (Main Program)
"FB_MixingSequencer_DB"(
    Act_MixTank_Temp := "AI_MixTank_TT01",      // Your analog input tag
    Act_MixTank_Weight := "AI_MixTank_WT01",
    Act_Agitator_RPM := "AI_Agitator_Speed",
    Act_HighShear_RPM := "AI_HighShear_Speed",
    Act_Hopper_Weight := "AI_Hopper_WT01",
    Act_Brix := "AI_Brix_Sensor",
    Act_PH := "AI_PH_Sensor",
    SP_Agitator_RPM => "AO_Agitator_SP",        // Your analog output tag
    SP_HighShear_RPM => "AO_HighShear_SP",
    SP_Temperature => "AO_Temperature_SP"
);

```

Note: Replace "AI_xxx" and "AO_xxx" with your actual I/O tag names from the TIA Portal hardware configuration.

PART B: Node-RED Configuration

B1. Install Required Nodes

```

cd ~/.node-red
npm install node-red-contrib-s7
npm install node-red-contrib-amqp

```

B2. S7 Connection Configuration

In Node-RED, create an S7 connection: | Setting | Value | | Host |
 192.168.1.1 (your PLC) | | Port | 102 | | Rack | 0 | | Slot | 1 | | Cycle Time | 500 ms |

B3. S7 Variable Mapping (Read from PLC → Publish to RabbitMQ)

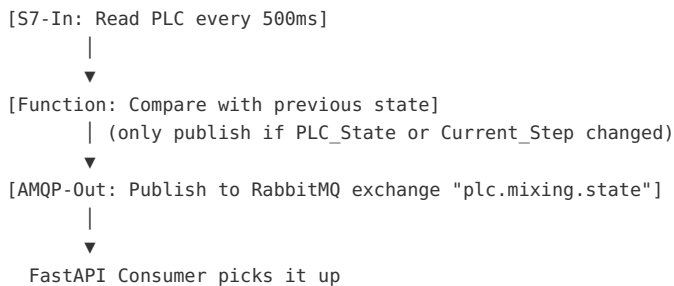
Create an **S7-In** node that reads these DB1780 addresses every 500ms:

Variable Name	S7 Address	Type
PLC_State	DB1780, INT18	INT
Current_Process	DB1780, INT20	INT
Current_Step	DB1780, INT22	INT

Current_Step_Flat	DB1780,INT24	INT
Step_Timer_Act	DB1780,INT26	INT
Step_Complete	DB1780,X28.0	BOOL
Batch_Complete	DB1780,X28.1	BOOL
Error_Code	DB1780,INT30	INT
MixTank_Temp_Act	DB1780,REAL32	REAL
MixTank_Weight_Act	DB1780,REAL36	REAL
Agitator_Act	DB1780,REAL40	REAL
HighShear_Act	DB1780,REAL44	REAL
Hopper_Weight_Act	DB1780,REAL48	REAL
Brix_Act	DB1780,REAL52	REAL
PH_Act	DB1780,REAL56	REAL
BatchID	DB1780,S34.20	STRING

⚠ **Important:** The exact byte offsets above are **approximate**. After you create DB1780 in TIA Portal with `Optimized Access = FALSE`, open the DB and look at the **Offset** column. Use those exact values in Node-RED.

B4. Node-RED Flow Logic



Function Node (change detection):

```

// Store previous state in flow context
var prev = flow.get('plc_state') || {};
var cur = msg.payload;

// Only publish if something changed
if (prev.PLC_State !== cur.PLC_State ||
    prev.Current_Step_Flat !== cur.Current_Step_Flat ||
    prev.Step_Complete !== cur.Step_Complete) {

    flow.set('plc_state', cur);
    msg.payload = {
        PLC_State: cur.PLC_State,
        Current_Process: cur.Current_Process,
        Current_Step: cur.Current_Step,
        Current_Step_Flat: cur.Current_Step_Flat,
        Step_Timer_Act: cur.Step_Timer_Act,
        Step_Complete: cur.Step_Complete,
        Batch_Complete: cur.Batch_Complete,
        BatchID: cur.BatchID,
        MixTank_Temp_Act: cur.MixTank_Temp_Act,
        MixTank_Weight_Act: cur.MixTank_Weight_Act,
    }
}

```

```

        Agitator_Act: cur.Agitator_Act,
        HighShear_Act: cur.HighShear_Act,
        timestamp: new Date().toISOString()
    };
    return msg;
}
return null; // Don't publish if nothing changed

```

B5. Writing Recipe TO PLC (PC → Node-RED → PLC)

Create an **HTTP-In** node (POST /api/plc/write-recipe) that accepts the JSON recipe from FastAPI and writes it to DB1780 using S7-Out nodes.

Flow:

```

[HTTP-In: POST /api/plc/write-recipe]
  |
  ▼
[Function: Map JSON to S7 writes]
  |
  ▼
[S7-Out: Write header fields]
[S7-Out: Write Process[0].Steps[0..7]]
[S7-Out: Write Process[1].Steps[0..7]]
... (loop through all active processes)
  |
  ▼
[S7-Out: Write RecipeReady = TRUE]
  |
  ▼
[HTTP-Response: 200 OK]

```

B6. Heartbeat Relay (PC → PLC)

```

[AMQP-In: Subscribe "pc.heartbeat"]
  |
  ▼
[S7-Out: Write DB1780.PC_Heartbeat (toggle TRUE/FALSE)]

```

PART C: Production Deployment Checklist

Step 1: PLC Setup (TIA Portal)

- ☐ Create UDT_ProcessStep and UDT_Process in TIA Portal
- ☐ Create DB1780 with Optimized Access = FALSE
- ☐ Create FB1780 with the sequencer logic
- ☐ Call FB1780 in OB1 with correct I/O mappings
- ☐ Download to PLC and verify DB1780 appears in the watch table
- ☐ Note the **exact byte offsets** from TIA Portal's DB view

Step 2: Node-RED Setup

- ☐ Install node-red-contrib-s7 and node-red-contrib-amqp
- ☐ Configure S7 connection to PLC IP (test with a simple read first)
- ☐ Map the S7 variables using exact offsets from Step 1
- ☐ Create the polling flow (read every 500ms)
- ☐ Create the change-detection function node
- ☐ Create the RabbitMQ publish node
- ☐ Create the recipe-write HTTP endpoint
- ☐ Create the heartbeat relay flow
- ☐ **Test:** Read PLC_State from Node-RED debug panel

Step 3: FastAPI Backend

- ☐ Update POST /plc/send-recipe/{batch_id} to POST the JSON to Node-RED's HTTP endpoint instead of returning it
- ☐ Add RabbitMQ consumer to listen for plc.mixing.state events
- ☐ On each event, write to mixing_batch_step_log table
- ☐ Add WebSocket broadcast to push state to frontend
- ☐ Add heartbeat publisher (toggle pc.heartbeat every 1s)

Step 4: Frontend (Already Done)

- ☒ x62-MixingControlV2.vue is stateless
- ☒ Reacts to plantData telemetry updates
- ☒ Recovery on mount works
- ☒ downloadRecipeToPlc() calls the backend API

Step 5: Integration Test

- ☐ Load a test batch on the PLC via the UI
 - ☐ Verify recipe appears in DB1780 watch table
 - ☐ Click START → verify PLC_State changes to 1
 - ☐ Watch steps auto-advance in the UI
 - ☐ **Pull-the-plug test:** Kill FastAPI at Step 5, restart, verify UI recovers at the correct step
 - ☐ **Manual step test:** Verify PLC pauses at ActionCode 21010 and waits for operator confirmation
-

PART D: Action Code Reference for PLC Logic

The PLC sequencer needs to know which steps are automatic and which require manual intervention:

Action Code	Description	PLC Behavior
10010	Setup / Initialize	Auto — apply setpoints, no wait
10020	Heating	Auto — wait for temp + time
10030	Mixing / Blending	Auto — run agitator for time
10040	Pasteurize	Auto — hold temp for time
21010	Manual Add to Mix Tank	MANUAL — PLC waits for PC
30020	Manual Transfer	MANUAL — PLC waits for PC

For all 21010 and 30020 steps, the PLC sets `PLC_State = 2 (WaitManual)` and holds position until the PC sends `StartCmd = TRUE` after the operator confirms.

PART E: Network Requirements

Device	IP Address	Port	Protocol
PLC	192.168.1.1	102	S7 (TCP)
Node-RED	192.168.1.100	1880	HTTP
RabbitMQ	192.168.1.100	5672	AMQP
FastAPI	192.168.1.100	8023	HTTP
Nuxt	192.168.1.100	3000	HTTP/WS

All middleware (Node-RED, RabbitMQ, FastAPI, Nuxt) can run on a **single industrial PC**. Only the PLC needs a separate Ethernet connection.

PART F: Safety Considerations

1. **Emergency Stop:** The PLC's hardware E-Stop circuit must be independent of this software. The `AbortCmd` is a software-level stop only.
2. **Heartbeat Timeout:** If `PC_Heartbeat` stops for 5 seconds, the PLC continues automatic steps but pauses manual steps. This prevents product loss.
3. **Recipe Validation:** Before setting `RecipeReady = TRUE`, the PLC should verify `ProcessCount > 0` and `BatchID` is not empty.
4. **Watchdog:** Node-RED should monitor its own S7 connection. If the S7 link drops, it should publish an alarm to RabbitMQ so FastAPI can alert the operator.